

GUV_circGLMM_fit

September 9, 2026

0.1 # A Hierarchical Model of Honeybee Waggle-Dances

0.2 Details

AUTHOR: James Foster 2025 03 14

MODIFIED: James Foster 2026 05 22

DESCRIPTION: Fit a hierarchical maximum-likelihood von Mises to honeybee waggle-run angles. Modified from [beetles.ipynb](#) from Jake Graving's `unwrap` package.

INPUTS: `colour_dance_reorg.csv`

OUTPUTS: `colour_dance_reorg.feather`, `GUV_unwrap_*.nc`, `model_comparison.pdf`

REFERENCES: - Sayin S, ... Graving JM, *et al.* (2025) The behavioral mechanisms governing collective motion in swarming locusts. *Science* 387,995-1000 DOI:[10.1126/science.adq7832](#)

- Graving JM, Sayin S, Costelloe BR, Goldshtein A & Foster JJ. Unwrapping circular statistics with Bayesian multilevel models.
- Abril-Pla, O., Andreani, V., Carroll, C., Dong, L., Fonnesbeck, C. J., Kochurov, M., Kumar, R., Lao, J., Luhmann, C. C., Martin, O. A., et al. (2023). PyMC: a modern, and comprehensive probabilistic programming framework in Python. *PeerJ Computer Science* 9, e1516.
- Capretto, T., Piho, C., Kumar, R., Westfall, J., Yarkoni, T., & Martin, O. (2022) **Bambi**: A simple interface for fitting Bayesian linear models in Python. *Journal of Statistical Software* 103(15), 1-29. doi:10.18637/jss.v103.i15
- Seyboldt, A., Carlson, E. L., & Carpenter, B. (2026). Preconditioning Hamiltonian Monte Carlo by minimizing Fisher Divergence. *arXiv preprint arXiv:2603.18845*.
- Magnusson, M., Andersen, M.R., Jonasson, J., and Vehtari, A. (2020). Leave-One-Out Cross-Validation for Bayesian Model Comparison in Large Data. *arXiv* doi:10.48550/arXiv.2001.00980

1 First set up the required packages

Don't forget to make sure the the kernel is running in a `conda` environment.

e.g. `conda create -n GUV-env python=3.12 -c conda-forge -y`

To use `g++` (recommended for speed) make sure `m2w64-toolchain` is installed on Windows.

```
[1]: import sys

# 20260520 - Modified for nutpie compliance with strict NumPy < 2 constraints
requirements = """
numpy>=1.26.4,<2.0.0
pandas==2.2.2
scipy==1.13.1
arviz==0.19.0
bambi==0.14.0
pymc==5.16.2
matplotlib==3.8.4
formulae==0.5.4
numba>=0.59.1,<=0.60.0
nutpie
netcdf4>=1.6.5,<1.7.0
h5py>=3.10.0,<4.0.0
git+https://github.com/jgraving/unwrap.git
"""

# Write it to a file
with open("nutpie_unwrap_requirements.txt", "w") as f:
    f.write(requirements.strip() + "\n")

# Install everything from the file (on Windows "{sys.executable}" needs to be
↳ in quotes)
!"{sys.executable}" -m pip install -r nutpie_unwrap_requirements.txt --no-cache

%set_env PYTENSOR_FLAGS=blas__ldflags='-llapack -lblas -lcbblas'
```

```
Collecting git+https://github.com/jgraving/unwrap.git (from -r
nutpie_unwrap_requirements.txt (line 13))
  Cloning https://github.com/jgraving/unwrap.git to
/private/var/folders/nq/smdrxv810_z_tfcmgmg5gkb00000gn/T/pip-req-build-ld8_xitk
  Running command git clone --filter=blob:none --quiet
https://github.com/jgraving/unwrap.git
/private/var/folders/nq/smdrxv810_z_tfcmgmg5gkb00000gn/T/pip-req-build-ld8_xitk
  Resolved https://github.com/jgraving/unwrap.git to commit
3e701175b665a12f4a0dcda6643aef9ea4b7aecf
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: numpy<2.0.0,>=1.26.4 in
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from -r
nutpie_unwrap_requirements.txt (line 1)) (1.26.4)
Requirement already satisfied: pandas==2.2.2 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from -r nutpie_unwrap_requirements.txt (line
2)) (2.2.2)
Requirement already satisfied: scipy==1.13.1 in /opt/miniconda3/envs/nutpie-
```

env/lib/python3.12/site-packages (from -r nutpie_unwrap_requirements.txt (line 3)) (1.13.1)

Requirement already satisfied: arviz==0.19.0 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from -r nutpie_unwrap_requirements.txt (line 4)) (0.19.0)

Requirement already satisfied: bambi==0.14.0 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from -r nutpie_unwrap_requirements.txt (line 5)) (0.14.0)

Requirement already satisfied: pymc==5.16.2 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from -r nutpie_unwrap_requirements.txt (line 6)) (5.16.2)

Requirement already satisfied: matplotlib==3.8.4 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from -r nutpie_unwrap_requirements.txt (line 7)) (3.8.4)

Requirement already satisfied: formulae==0.5.4 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from -r nutpie_unwrap_requirements.txt (line 8)) (0.5.4)

Requirement already satisfied: numba<=0.60.0,>=0.59.1 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from -r nutpie_unwrap_requirements.txt (line 9)) (0.60.0)

Requirement already satisfied: nutpie in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from -r nutpie_unwrap_requirements.txt (line 10)) (0.13.2)

Requirement already satisfied: netcdf4<1.7.0,>=1.6.5 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from -r nutpie_unwrap_requirements.txt (line 11)) (1.6.5)

Requirement already satisfied: h5py<4.0.0,>=3.10.0 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from -r nutpie_unwrap_requirements.txt (line 12)) (3.16.0)

Requirement already satisfied: python-dateutil>=2.8.2 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from pandas==2.2.2->-r nutpie_unwrap_requirements.txt (line 2)) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from pandas==2.2.2->-r nutpie_unwrap_requirements.txt (line 2)) (2026.2)

Requirement already satisfied: tzdata>=2022.7 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from pandas==2.2.2->-r nutpie_unwrap_requirements.txt (line 2)) (2026.2)

Requirement already satisfied: setuptools>=60.0.0 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from arviz==0.19.0->-r nutpie_unwrap_requirements.txt (line 4)) (82.0.1)

Requirement already satisfied: packaging in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from arviz==0.19.0->-r nutpie_unwrap_requirements.txt (line 4)) (26.2)

Requirement already satisfied: dm-tree>=0.1.8 in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from arviz==0.19.0->-r nutpie_unwrap_requirements.txt (line 4)) (0.1.10)

Requirement already satisfied: xarray>=2022.6.0 in /opt/miniconda3/envs/nutpie-

```

env/lib/python3.12/site-packages (from arviz==0.19.0->-r
nutpie_unwrap_requirements.txt (line 4)) (2026.4.0)
Requirement already satisfied: h5netcdf>=1.0.2 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from arviz==0.19.0->-r
nutpie_unwrap_requirements.txt (line 4)) (1.8.1)
Requirement already satisfied: typing-extensions>=4.1.0 in
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from
arviz==0.19.0->-r nutpie_unwrap_requirements.txt (line 4)) (4.15.0)
Requirement already satisfied: xarray-einstats>=0.3 in
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from
arviz==0.19.0->-r nutpie_unwrap_requirements.txt (line 4)) (0.9.1)
Requirement already satisfied: graphviz in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from bambi==0.14.0->-r
nutpie_unwrap_requirements.txt (line 5)) (0.21)
Requirement already satisfied: cachetools>=4.2.1 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from pymc==5.16.2->-r
nutpie_unwrap_requirements.txt (line 6)) (7.1.3)
Requirement already satisfied: cloudpickle in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from pymc==5.16.2->-r
nutpie_unwrap_requirements.txt (line 6)) (3.1.2)
Requirement already satisfied: pytensor<2.26,>=2.25.1 in
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from
pymc==5.16.2->-r nutpie_unwrap_requirements.txt (line 6)) (2.25.5)
Requirement already satisfied: rich>=13.7.1 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from pymc==5.16.2->-r
nutpie_unwrap_requirements.txt (line 6)) (15.0.0)
Requirement already satisfied: threadpoolctl<4.0.0,>=3.1.0 in
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from
pymc==5.16.2->-r nutpie_unwrap_requirements.txt (line 6)) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from matplotlib==3.8.4->-r
nutpie_unwrap_requirements.txt (line 7)) (1.3.3)
Requirement already satisfied: cycycler>=0.10 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from matplotlib==3.8.4->-r
nutpie_unwrap_requirements.txt (line 7)) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from matplotlib==3.8.4->-r
nutpie_unwrap_requirements.txt (line 7)) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from matplotlib==3.8.4->-r
nutpie_unwrap_requirements.txt (line 7)) (1.5.0)
Requirement already satisfied: pillow>=8 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from matplotlib==3.8.4->-r
nutpie_unwrap_requirements.txt (line 7)) (12.2.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from matplotlib==3.8.4->-r
nutpie_unwrap_requirements.txt (line 7)) (3.3.2)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in

```

```

/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from
numba<=0.60.0,>=0.59.1->-r nutpie_unwrap_requirements.txt (line 9)) (0.43.0)
Requirement already satisfied: cftime in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from netcdf4<1.7.0,>=1.6.5->-r
nutpie_unwrap_requirements.txt (line 11)) (1.6.5)
Requirement already satisfied: certifi in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from netcdf4<1.7.0,>=1.6.5->-r
nutpie_unwrap_requirements.txt (line 11)) (2026.4.22)
Requirement already satisfied: filelock>=3.15 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from pytensor<2.26,>=2.25.1->pymc==5.16.2->-r
nutpie_unwrap_requirements.txt (line 6)) (3.29.0)
Requirement already satisfied: etuples in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from pytensor<2.26,>=2.25.1->pymc==5.16.2->-r
nutpie_unwrap_requirements.txt (line 6)) (0.3.10)
Requirement already satisfied: logical-unification in
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from
pytensor<2.26,>=2.25.1->pymc==5.16.2->-r nutpie_unwrap_requirements.txt (line
6)) (0.4.7)
Requirement already satisfied: miniKanren in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from pytensor<2.26,>=2.25.1->pymc==5.16.2->-r
nutpie_unwrap_requirements.txt (line 6)) (1.0.5)
Requirement already satisfied: cons in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from pytensor<2.26,>=2.25.1->pymc==5.16.2->-r
nutpie_unwrap_requirements.txt (line 6)) (0.4.7)
Requirement already satisfied: pyarrow>=12.0.0 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from nutpie->-r nutpie_unwrap_requirements.txt
(line 10)) (24.0.0)
Requirement already satisfied: absl-py>=0.6.1 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from dm-tree>=0.1.8->arviz==0.19.0->-r
nutpie_unwrap_requirements.txt (line 4)) (2.4.0)
Requirement already satisfied: attrs>=18.2.0 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from dm-tree>=0.1.8->arviz==0.19.0->-r
nutpie_unwrap_requirements.txt (line 4)) (26.1.0)
Requirement already satisfied: wrapt>=1.11.2 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from dm-tree>=0.1.8->arviz==0.19.0->-r
nutpie_unwrap_requirements.txt (line 4)) (2.1.2)
Requirement already satisfied: six>=1.5 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from python-dateutil>=2.8.2->pandas==2.2.2->-r
nutpie_unwrap_requirements.txt (line 2)) (1.17.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from
rich>=13.7.1->pymc==5.16.2->-r nutpie_unwrap_requirements.txt (line 6)) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from
rich>=13.7.1->pymc==5.16.2->-r nutpie_unwrap_requirements.txt (line 6)) (2.20.0)
Requirement already satisfied: mdurl~0.1 in /opt/miniconda3/envs/nutpie-
env/lib/python3.12/site-packages (from markdown-it-
py>=2.2.0->rich>=13.7.1->pymc==5.16.2->-r nutpie_unwrap_requirements.txt (line

```

6)) (0.1.2)

Requirement already satisfied: toolz in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from logical-unification->pytensor<2.26,>=2.25.1->pymc==5.16.2->-r nutpie_unwrap_requirements.txt (line 6)) (1.1.0)

Requirement already satisfied: multipledispatch in /opt/miniconda3/envs/nutpie-env/lib/python3.12/site-packages (from logical-unification->pytensor<2.26,>=2.25.1->pymc==5.16.2->-r nutpie_unwrap_requirements.txt (line 6)) (1.0.0)

env: PYTENSOR_FLAGS=blas__ldflags='-llapack -lblas -lcblas'

Report packages installed in enviroment.

```
[2]: !"{sys.executable}" -m pip list
```

Package	Version
absl-py	2.4.0
anyio	4.13.0
appnope	0.1.4
argon2-cffi	25.1.0
argon2-cffi-bindings	25.1.0
arrow	1.4.0
arviz	0.19.0
asttokens	3.0.1
async-lru	2.3.0
attrs	26.1.0
babel	2.18.0
backports.zstd	1.5.0
bambi	0.14.0
beautifulsoup4	4.14.3
bleach	6.3.0
Brotli	1.2.0
cached-property	1.5.2
cachetools	7.1.3
certifi	2026.4.22
cffi	2.0.0
cftime	1.6.5
charset-normalizer	3.4.7
cloudpickle	3.1.2
comm	0.2.3
cons	0.4.7
contourpy	1.3.3
cycler	0.12.1
debugpy	1.8.20
decorator	5.3.1
defusedxml	0.7.1
dm-tree	0.1.10
entrypoints	0.4

etuples	0.3.10
exceptiongroup	1.3.1
executing	2.2.1
fastjsonschema	2.21.2
filelock	3.29.0
fonttools	4.63.0
formulae	0.5.4
fqdn	1.5.1
graphviz	0.21
h11	0.16.0
h2	4.3.0
h5netcdf	1.8.1
h5py	3.16.0
hpack	4.1.0
httpcore	1.0.9
httpx	0.28.1
hyperframe	6.1.0
idna	3.15
importlib_metadata	8.8.0
importlib_resources	7.1.0
ipykernel	6.29.5
ipython	9.13.0
ipython_pygments_lexers	1.1.1
isoduration	20.11.0
jedi	0.19.2
Jinja2	3.1.6
json5	0.14.0
jsonpointer	3.1.1
jsonschema	4.26.0
jsonschema-specifications	2025.9.1
jupyter_client	7.4.9
jupyter_core	5.9.1
jupyter-events	0.12.1
jupyter-lsp	2.3.1
jupyter_server	2.18.2
jupyter_server_terminals	0.5.4
jupyterlab	4.5.7
jupyterlab_pygments	0.3.0
jupyterlab_server	2.28.0
kiwisolver	1.5.0
lark	1.3.1
llvmlite	0.43.0
logical-unification	0.4.7
markdown-it-py	4.2.0
MarkupSafe	3.0.3
matplotlib	3.8.4
matplotlib-inline	0.2.2
mdurl	0.1.2

miniKanren	1.0.5
mistune	3.2.1
multiplatformdispatch	1.0.0
nbclient	0.10.4
nbconvert	7.17.1
nbformat	5.10.4
nest_asyncio	1.6.0
netCDF4	1.6.5
notebook	7.5.6
notebook_shim	0.2.4
numba	0.60.0
numpy	1.26.4
nutpie	0.13.2
overrides	7.7.0
packaging	26.2
pandas	2.2.2
pandocfilters	1.5.0
parso	0.8.7
pexpect	4.9.0
pillow	12.2.0
pip	26.1.1
platformdirs	4.9.6
prometheus_client	0.25.0
prompt_toolkit	3.0.52
psutil	7.2.2
ptyprocess	0.7.0
pure_eval	0.2.3
pyarrow	24.0.0
pycparser	3.0
Pygments	2.20.0
pymc	5.16.2
pyobjc-core	12.1
pyobjc-framework-Cocoa	12.1
pyparsing	3.3.2
PySocks	1.7.1
pytensor	2.25.5
python-dateutil	2.9.0.post0
python-json-logger	3.2.1
pytz	2026.2
PyYAML	6.0.3
pyzmq	24.0.1
referencing	0.37.0
requests	2.34.2
rfc3339_validator	0.1.4
rfc3986-validator	0.1.1
rfc3987-syntax	1.1.0
rich	15.0.0
rpds-py	0.30.0

scipy	1.13.1
Send2Trash	2.1.0
setuptools	82.0.1
six	1.17.0
sniffio	1.3.1
soupsieve	2.8.3
stack_data	0.6.3
terminado	0.18.1
threadpoolctl	3.6.0
tinycss2	1.4.0
tomli	2.4.1
toolz	1.1.0
tornado	6.5.5
traitlets	5.15.0
typing_extensions	4.15.0
typing_utils	0.1.0
tzdata	2026.2
unwrap	0.0.1
uri-template	1.3.0
urllib3	2.7.0
wcwidth	0.7.0
webcolors	25.10.0
webencodings	0.5.1
websocket-client	1.9.0
wheel	0.47.0
wrapt	2.1.2
xarray	2026.4.0
xarray-einstats	0.9.1
zipp	4.1.0

2 Set up custom functions

Create a function for plotting circular model output in degrees clockwise.

```
[3]: import matplotlib.pyplot as plt
import arviz as az

def plot_circular_trace(idata,
                        vars,
                        cvars,
                        cvar_units='degrees',
                        save_trans=False):
    """
    Transforms circular posterior variables from radians to degrees (inverted
    for clockwise plotting) and plots the ArviZ trace.

    Parameters:
```

```

-----
idata : arviz.InferenceData
    The ArviZ InferenceData object containing the posterior samples.
vars : list of str
    The full list of variable names to plot in the trace.
cvars : list of str
    The subset of variable names that are circular and need transformation.
cvar_units : str, optional
    Units for the circular variables (e.g., 'degrees' or 'radians').
↳ Default is 'degrees'.
save_trans : bool, optional
    Whether to return the transformed InferenceData object. Default is True.

Returns:
-----
idata_trans : arviz.InferenceData or None
    A copy of the InferenceData object with transformed circular variables.
↳ if save_trans=True,
    otherwise None.
"""
# Transform the circular data for interpretability
idata_trans = idata.copy() # set up a variable for transformed data

if cvar_units == 'degrees':
    for cvar in cvars:
        # Convert each to degrees and invert to plot clockwise
        idata_trans.posterior[cvar] = -180 * idata_trans.posterior[cvar] /
↳ np.pi

# Generate the trace plot
az.plot_trace(
    idata_trans,
    var_names=vars,
    circ_var_names=cvars,
    circ_var_units=cvar_units
)

plt.tight_layout()
plt.show()

if save_trans:
    return idata_trans

return None

```

3 Prepare the data

3.1 Import and format the data

Load the preprocessed data from CSV and convert to radians. Relabel the two conditions in each factor to binary.

```
[4]: import pandas as pd
import numpy as np
from unwrap.link.numpy import shifted_modulo

# Load the file
file_path = '1Data/colour_dance_reorg.csv' #JJF windows is having trouble
    ↳ finding this file
df = pd.read_csv(file_path, sep=',')

# Create a binary indicator for the light intensity (1 for 'low', 0 for 'high')
df['B'] = df['brightn'].apply(lambda x: 1 if 'l' in x else 0)
# Create a binary indicator for the light colour (1 for 'UV', 0 for 'green')
df['C'] = df['colour'].apply(lambda x: 1 if 'u' in x else 0)
# Rename sun azimuth to use capitalised label
df['SunAz'] = df['sun_az_rad']
# Rename run to use capitalised label
df['Run'] = df['run']

# Convert the dance angle column to the correct format (already radians?)
df['angle'] = pd.to_numeric(df['angle'], errors='coerce') # Convert
    ↳ non-numeric values (e.g., 'NA') to NaN

# Normalise all angles to be between -pi and pi (should be already)
df['angle'] = df['angle'].apply(lambda x: shifted_modulo(x) if pd.notnull(x)
    ↳ else x)
df['sun_az_rad'] = df['sun_az_rad'].apply(lambda x: shifted_modulo(x) if pd.
    ↳ notnull(x) else x)

# Keep only the relevant columns and drop rows with missing values
# df = df[['angle', 'ID', 'B', 'C', 'Run', 'SunAz']].dropna().copy()
df = df[['angle', 'ID', 'B', 'C', 'Run', 'SunAz']].copy() # we don't need to drop
    ↳ NA as we're not using SunAz
df.reset_index(drop=True, inplace=True)

# Rename columns: change 'ID' to 'Individual'
df = df.rename(columns={'ID': 'Individual'})
df['Individual'] = df['Individual'].astype('category')

# Simplify to codes
df['Individual'] = df['Individual'].cat.codes + 1
# save to align with raw data in R
```

```
df.to_feather('1Data/colour_dance_reorg.feather')

df.head()
```

```
[4]:      angle  Individual  B  C  Run      SunAz
0  1.570796           3  1  1    1 -2.647205
1 -0.087266           3  1  1    2 -2.647205
2 -0.349066           3  1  1    3 -2.647205
3  0.698132           3  1  1    4 -2.647205
4 -0.349066           3  1  1    5 -2.647205
```

3.2 Inspect the data

Plot the data from the 1st 6 dances. *N.B* Pandas indexes categories in alphabetical order, whereas R indexes them in order of appearance. We need to extract the Individual column from colour_dance_reorg.feather to align them.

```
[5]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import circmean, circstd
from matplotlib.lines import Line2D

# Helper function to compute the circular median.
def circ_median(angles):
    angles = np.sort(angles)
    diffs = np.abs(np.angle(np.exp(1j * (angles[:, None] - angles[None, :]))))
    sum_diffs = np.sum(diffs, axis=1)
    return angles[np.argmin(sum_diffs)]

# Helper function to scatter-plot data for one condition.
def plot_scatter(ax, data, angle_col, radial_col, color, label):
    if not data.empty:
        ax.scatter(data[angle_col], data[radial_col],
                    color=color, alpha=0.6, label=label)

# Helper function to compute and plot summary statistics.
# It plots a solid vertical line at the circular mean and a dashed vertical
↳ line at the circular median,
# both in the specified color. It also fills the region ± the circular standard
↳ deviation.
def plot_summary_stats(ax, angles, radial_max, color):
    # Compute circular mean, median, and standard deviation.
    mean_val = circmean(angles, high=np.pi, low=-np.pi)
    median_val = circ_median(angles)
    std_val = circstd(angles, high=np.pi, low=-np.pi, normalize=True)
```

```

# Plot mean (solid) and median (dashed).
ax.plot([mean_val, mean_val], [0, radial_max], color=color, linewidth=2)
ax.plot([median_val, median_val], [0, radial_max], color=color,
↳linewidth=2, linestyle='--')

# Fill the region between mean minus std and mean plus std.
theta_vals = np.linspace(mean_val - std_val, mean_val + std_val, 100)
r_vals = np.linspace(0, radial_max, 100)
Theta, R = np.meshgrid(theta_vals, r_vals)
ax.contourf(Theta, R, np.ones_like(Theta), alpha=0.05, colors=[color]) #↳
↳was alpha = 0.2, but covers too much data

return mean_val, median_val, std_val

# Get unique individual IDs.
unique_individuals = df['Individual'].unique()
plot_individuals = unique_individuals[range(0,6)] # for now just the 1st 6

# Create a grid of polar subplots.
n_plots = len(plot_individuals)
n_rows, n_cols = ( np.intc(n_plots/2) , 2 ) # Adjust grid dimensions as needed.
fig, axes = plt.subplots(n_rows, n_cols, subplot_kw={'projection': 'polar'},
↳figsize=(18, np.intc(n_plots*3)))
axes = axes.flatten()

radial_max = np.max(df['Run']) # Maximum radial value (maximum runs)

# Plot for each individual.
for idx, individual in enumerate(plot_individuals):
    ax = axes[idx]
    # Set axes to geographic format (clockwise, 0° at the top)
    ax.set_theta_direction(-1) #reverse to clockwise
    ax.set_theta_zero_location('N') #place 0° at the top
    individual_data = df[df['Individual'] == individual]

    # Separate Light bright and Light dim data (drop NaNs in angle).
    data_on = individual_data[individual_data['B'] == 0].
    ↳dropna(subset=['angle'])
    data_off = individual_data[individual_data['B'] == 1].
    ↳dropna(subset=['angle'])
    # Separate bright data.
    data_gh = data_on[data_on['C'] == 0]
    data_uh = data_on[data_on['C'] == 1]
    # Separate dim data.
    data_gl = data_off[data_off['C'] == 0]
    data_ul = data_off[data_off['C'] == 1]

```

```

# Scatter-plot data.
plot_scatter(ax, data_gh, 'angle', 'Run', '#00CD00', 'Green Bright') #
↳green3 hex
plot_scatter(ax, data_gl, 'angle', 'Run', 'darkgreen', 'Green Dim')
plot_scatter(ax, data_uh, 'angle', 'Run', '#CD00CD', 'UV Bright') #
↳magenta3 hex
plot_scatter(ax, data_ul, 'angle', 'Run', 'purple', 'UV Dim')

#Plot the sun's azimuth for reference
ax.plot(individual_data['SunAz'].values[range(0,2)], [0, radial_max],
↳color='orange', linewidth=1)
# Plot summary statistics for each condition.
if not data_gh.empty:
    plot_summary_stats(ax, data_gh['angle'].values, radial_max, '#00CD00')
if not data_gl.empty:
    plot_summary_stats(ax, data_gl['angle'].values, radial_max, 'darkgreen')
if not data_uh.empty:
    plot_summary_stats(ax, data_uh['angle'].values, radial_max, '#CD00CD')
if not data_ul.empty:
    plot_summary_stats(ax, data_ul['angle'].values, radial_max, 'purple')

ax.set_title(f'Dance {individual}', pad=30)
ax.set_ylim(0, radial_max + 1)
# Remove individual legends from each subplot.
ax.legend().set_visible(False)

# Create a shared custom legend.
legend_handles = [
    # For sun azimuth (orange)
    Line2D([0], [0], color = 'orange', lw = 1, label = 'Sun Azimuth (relative
↳to North)'),
    # For green bright (green3)
    Line2D([0], [0], marker='o', color='w', markerfacecolor='#00CD00',
↳markersize=8, label='Green Bright Data'),
    Line2D([0], [0], color='#00CD00', lw=2, label='Green Bright Mean'),
    Line2D([0], [0], color='#00CD00', lw=2, linestyle='--', label='Green Bright
↳Median'),
    # For green dim (darkgreen)
    Line2D([0], [0], marker='o', color='w', markerfacecolor='darkgreen',
↳markersize=8, label='Green Dim Data'),
    Line2D([0], [0], color='darkgreen', lw=2, label='Green Dim Mean'),
    Line2D([0], [0], color='darkgreen', lw=2, linestyle='--', label='Green Dim
↳Median'),
    # For UV bright (magenta3)
    Line2D([0], [0], marker='o', color='w', markerfacecolor='#CD00CD',
↳markersize=8, label='UV Bright Data'),

```

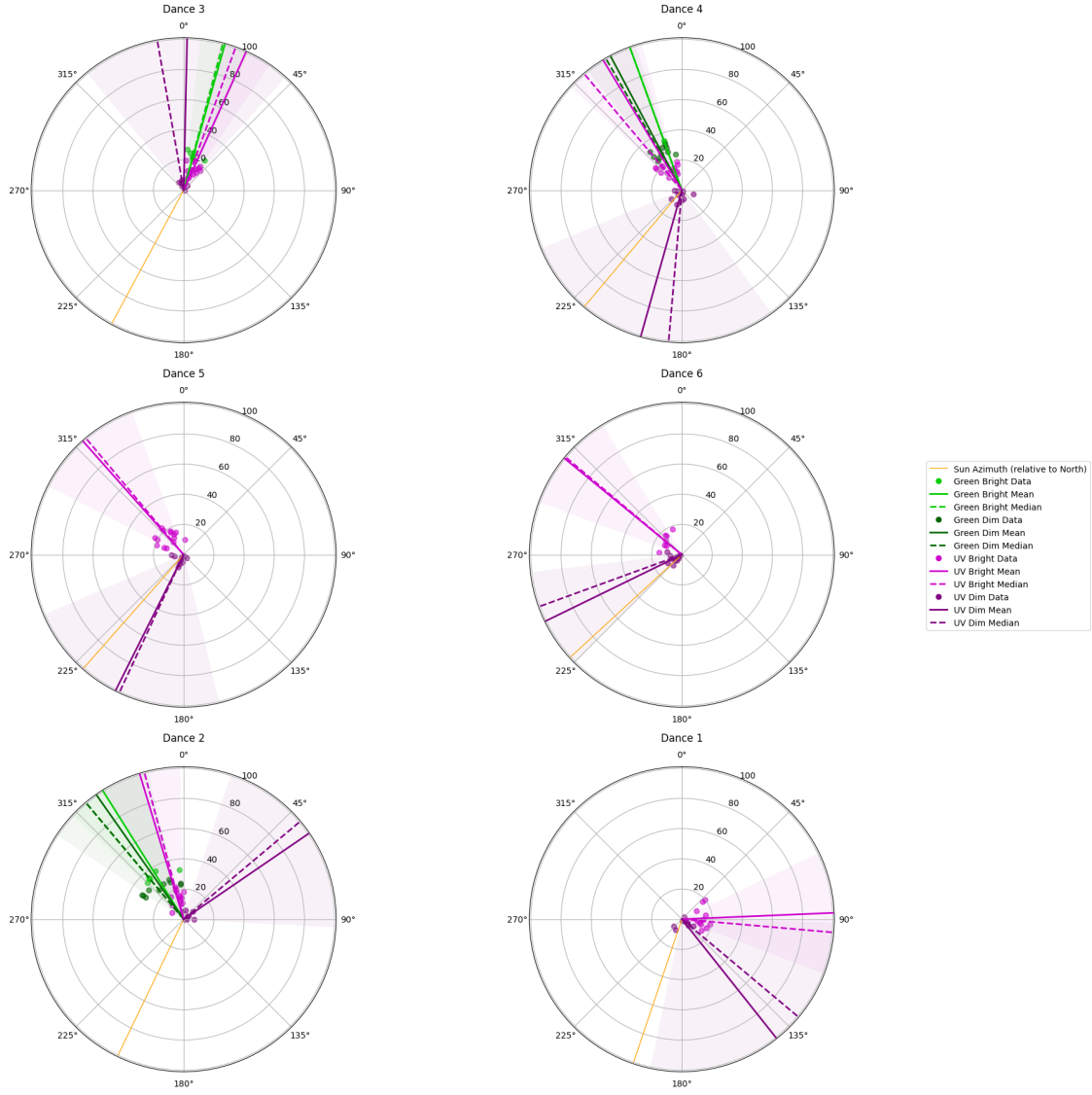
```

    Line2D([0], [0], color='#CD00CD', lw=2, label='UV Bright Mean'),
    Line2D([0], [0], color='#CD00CD', lw=2, linestyle='--', label='UV Bright_
↳Median'),
    # For UV dim (purple)
    Line2D([0], [0], marker='o', color='w', markerfacecolor='purple',
↳markersize=8, label='UV Dim Data'),
    Line2D([0], [0], color='purple', lw=2, label='UV Dim Mean'),
    Line2D([0], [0], color='purple', lw=2, linestyle='--', label='UV Dim_
↳Median'),
]

# Add one shared legend to the right of the figure.
fig.legend(handles=legend_handles, loc='center right', bbox_to_anchor=(1.10, 0.
↳5), fontsize='medium')

plt.tight_layout()
plt.show()

```



4 Fit a circular model with all potential effects

First fit the maximal model with effects of different individuals, light intensities and LED colours on both mean direction (μ) and directional consistency (κ).

$$\mu \sim B + C + (1 + B + C + B:C | \text{Individual})$$

$$\kappa \sim B + C + (1 + B + C + B:C | \text{Individual})$$

```
[ ]: import numpy as np
import pandas as pd
import bambi as bmb
import arviz as az
```



```

import matplotlib.pyplot as plt

from unwrap.link.bambi import LINKS
from unwrap.link.numpy import inverse_softplus
from unwrap.prior import CircularUniformPrior
from pymc import VonMises
from scipy.stats import vonmises, circstd

# Define the model formula with updated, capitalized variable names.
formula_interaction = bmb.Formula(
    "angle ~ B + C + B:C + (1 + B + C + B:C |Individual)",
    "kappa ~ B + C + B:C + (1 + B + C + B:C |Individual)",
)

# Specify priors for the model with capitalized keys and updated variable names.
priors_interaction = {
    # Fixed effects for :
    "Intercept": CircularUniformPrior(), # Test
    "B": bmb.Prior("VonMises", mu=0, kappa=0.5), # Test
    "C": bmb.Prior("VonMises", mu=0, kappa=0.5), # Test
    "B:C": bmb.Prior("VonMises", mu=0, kappa=0.5), # Test
    # Individual-level effects for : #bias to high kappa appears to cause
    ↪divergent transitions here; large sigma slows estimation; mu=log(30),
    ↪sigma=0.1 appears to be a good compromise.
    "1|Individual": bmb.Prior(
        "VonMises", mu=0, kappa=bmb.Prior("LogNormal", mu=np.log(30), sigma=0.
    ↪60) # Informative prior, individuals close to pop mean circsd ~10° # do not
    ↪change!
    ),
    "B|Individual": bmb.Prior(
        "VonMises", mu=0, kappa=bmb.Prior("LogNormal", mu=np.log(30), sigma=0.
    ↪60) # # Works
    ),
    "C|Individual": bmb.Prior(
        "VonMises", mu=0, kappa=bmb.Prior("LogNormal", mu=np.log(30), sigma=0.
    ↪60) # # Works
    ),
    "B:C|Individual": bmb.Prior(
        "VonMises", mu=0, kappa=bmb.Prior("LogNormal", mu=np.log(30), sigma=0.
    ↪60) # # Works
    ), #Test
    # Priors for the -model (inside the "kappa" dictionary):
    "kappa": {
        # Fixed effects for :
        "Intercept": bmb.Prior("Normal", mu=inverse_softplus(3), sigma=3.0), #
    ↪Works

```

```

    "B": bmb.Prior("Normal", mu=0, sigma=1.0), # Test
    "C": bmb.Prior("Normal", mu=0, sigma=1.0), # Test
    "B:C": bmb.Prior("Normal", mu=0, sigma=1.0), # Test
    # Individual-level effects for :
    "1|Individual": bmb.Prior(
        "Normal", mu=0, sigma=bmb.Prior("HalfStudentT", nu=3, sigma=1.0) #_
    ↪Works
    ), #Works
    "B|Individual": bmb.Prior(
        "Normal", mu=0, sigma=bmb.Prior("HalfStudentT", nu=3, sigma=1.0) #_
    ↪Works
    ),
    "C|Individual": bmb.Prior(
        "Normal", mu=0, sigma=bmb.Prior("HalfStudentT", nu=3, sigma=1.0) #_
    ↪Works
    ),
    "B:C|Individual": bmb.Prior(
        "Normal", mu=0, sigma=bmb.Prior("HalfStudentT", nu=3, sigma=1.0) #_
    ↪Works
    ),
}

}

# Build the model using the updated formula and priors.
model_interaction = bmb.Model(
    formula_interaction,
    df,
    family="vonmises",
    priors=priors_interaction,
    link={"mu": LINKS["shifted_modulo"], "kappa": LINKS["softplus"]},
    center_predictors=False,
    dropna=True,
)

print(model_interaction)

# Fit the model using Bambi's native nutpie interface
idata_interaction = model_interaction.fit(
    draws=2000,
    tune=1000,
    nuts_sampler="nutpie", #build and route via nutpie automatically
    target_accept=0.98
)

# Save model.
az.to_netcdf(idata_interaction, "2Results/GUV_unwrap_model.nc")

```

```

    Formula: angle ~ B + C + B:C + (1 + B + C + B:C |Individual)
           kappa ~ B + C + B:C + (1 + B + C + B:C |Individual)
    Family: vonmises
    Link: mu = shifted_modulo
           kappa = softplus
Observations: 6231
    Priors:
    target = mu
    Common-level effects
        Intercept ~ CircularUniform()
        B ~ VonMises(mu: 0.0, kappa: 0.5)
        C ~ VonMises(mu: 0.0, kappa: 0.5)
        B:C ~ VonMises(mu: 0.0, kappa: 0.5)

    Group-level effects
        1|Individual ~ VonMises(mu: 0.0, kappa: LogNormal(mu: 3.4012, sigma:
0.6))
        B|Individual ~ VonMises(mu: 0.0, kappa: LogNormal(mu: 3.4012, sigma:
0.6))
        C|Individual ~ VonMises(mu: 0.0, kappa: LogNormal(mu: 3.4012, sigma:
0.6))
        B:C|Individual ~ VonMises(mu: 0.0, kappa: LogNormal(mu: 3.4012,
sigma: 0.6))
    target = kappa
    Common-level effects
        kappa_Intercept ~ Normal(mu: 2.9489, sigma: 3.0)
        kappa_B ~ Normal(mu: 0.0, sigma: 1.0)
        kappa_C ~ Normal(mu: 0.0, sigma: 1.0)
        kappa_B:C ~ Normal(mu: 0.0, sigma: 1.0)

    Group-level effects
        kappa_1|Individual ~ Normal(mu: 0.0, sigma: HalfStudentT(nu: 3.0,
sigma: 1.0))
        kappa_B|Individual ~ Normal(mu: 0.0, sigma: HalfStudentT(nu: 3.0,
sigma: 1.0))
        kappa_C|Individual ~ Normal(mu: 0.0, sigma: HalfStudentT(nu: 3.0,
sigma: 1.0))
        kappa_B:C|Individual ~ Normal(mu: 0.0, sigma: HalfStudentT(nu: 3.0,
sigma: 1.0))

/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/pymc/sampling/mcmc.py:303: UserWarning: `var_names` are currently
ignored by the nutpie sampler
    warnings.warn(

<IPython.core.display.HTML object>

<nutpie.sample._BackgroundSampler at 0x19e8d7e90>

```

```
[ ]: '2Results/GUV_unwrap_model.nc'
```

4.1 Check convergence heuristics

```
[7]: #check convergence heuristics
az.summary(
    idata_interaction,
    var_names=["Intercept", #slightly right shifted, should converge to around 0.26
               "B",
               "C",
               "B:C",
               "kappa_Intercept",
               "kappa_B",
               "kappa_C",
               "kappa_B:C",
               ],
    circ_var_names=["Intercept",
                    "B",
                    "C",
                    "B:C",
                    ],
)
```

```
[7]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk \
Intercept	0.206	0.087	0.037	0.365	0.005	0.003	322.0
B	-0.542	0.126	-0.781	-0.309	0.003	0.003	1238.0
C	0.072	0.036	0.005	0.140	0.000	0.001	2251.0
B:C	-0.981	0.161	-1.290	-0.678	0.003	0.004	1013.0
kappa_Intercept	6.002	0.390	5.254	6.725	0.010	0.007	1683.0
kappa_B	-4.339	0.404	-5.114	-3.602	0.010	0.007	1661.0
kappa_C	2.726	0.386	2.011	3.454	0.009	0.006	1948.0
kappa_B:C	-2.731	0.443	-3.531	-1.876	0.010	0.007	1970.0

	ess_tail	r_hat
Intercept	611.0	1.02
B	1965.0	1.00
C	3589.0	1.00
B:C	1844.0	1.00
kappa_Intercept	3142.0	1.00
kappa_B	3063.0	1.00
kappa_C	3400.0	1.00
kappa_B:C	3423.0	1.00

```
[8]: #check convergence heuristics on random effects
az.summary(
    idata_interaction,
```

```

var_names=[
  "1|Individual_kappa",
  "B|Individual_kappa",
  "C|Individual_kappa",
  "B:C|Individual_kappa",
  "kappa_1|Individual_sigma",
  "kappa_B|Individual_sigma",
  "kappa_C|Individual_sigma",
  "kappa_B:C|Individual_sigma",
],
# filter_vars='like',
# circ_var_names=[]
)

```

```

[8]:
      mean      sd  hdi_3%  hdi_97%  mcse_mean  \
1|Individual_kappa      1.585   0.157   1.269   1.859      0.001
B|Individual_kappa      2.793   0.679   1.683   4.064      0.026
C|Individual_kappa     38.811  17.273  13.417  69.135      0.679
B:C|Individual_kappa      1.620   0.302   1.102   2.185      0.008
kappa_1|Individual_sigma      2.052   0.219   1.652   2.468      0.005
kappa_B|Individual_sigma      1.610   0.290   1.084   2.155      0.008
kappa_C|Individual_sigma      0.864   0.408   0.000   1.466      0.017
kappa_B:C|Individual_sigma      1.933   0.310   1.357   2.524      0.008

```

```

      mcse_sd  ess_bulk  ess_tail  r_hat
1|Individual_kappa      0.001  11151.0   5263.0   1.00
B|Individual_kappa      0.019    726.0   1122.0   1.01
C|Individual_kappa      0.480    620.0   1333.0   1.01
B:C|Individual_kappa      0.006   1400.0   2608.0   1.00
kappa_1|Individual_sigma      0.004   1786.0   4077.0   1.00
kappa_B|Individual_sigma      0.005   1476.0   2734.0   1.00
kappa_C|Individual_sigma      0.012    555.0    456.0   1.00
kappa_B:C|Individual_sigma      0.006   1666.0   2249.0   1.00

```

4.2 Visualise output of simple model

- The “Intercept” is the condition: **Green, Bright**
- B is the effect dimming the light: **Green, Dim**
- C is the effect of switching to UV: **UV, Bright**
- B:C is the effect of switching to UV-dim, compared to UV-bright and green-dim: **UV, Dim**

```

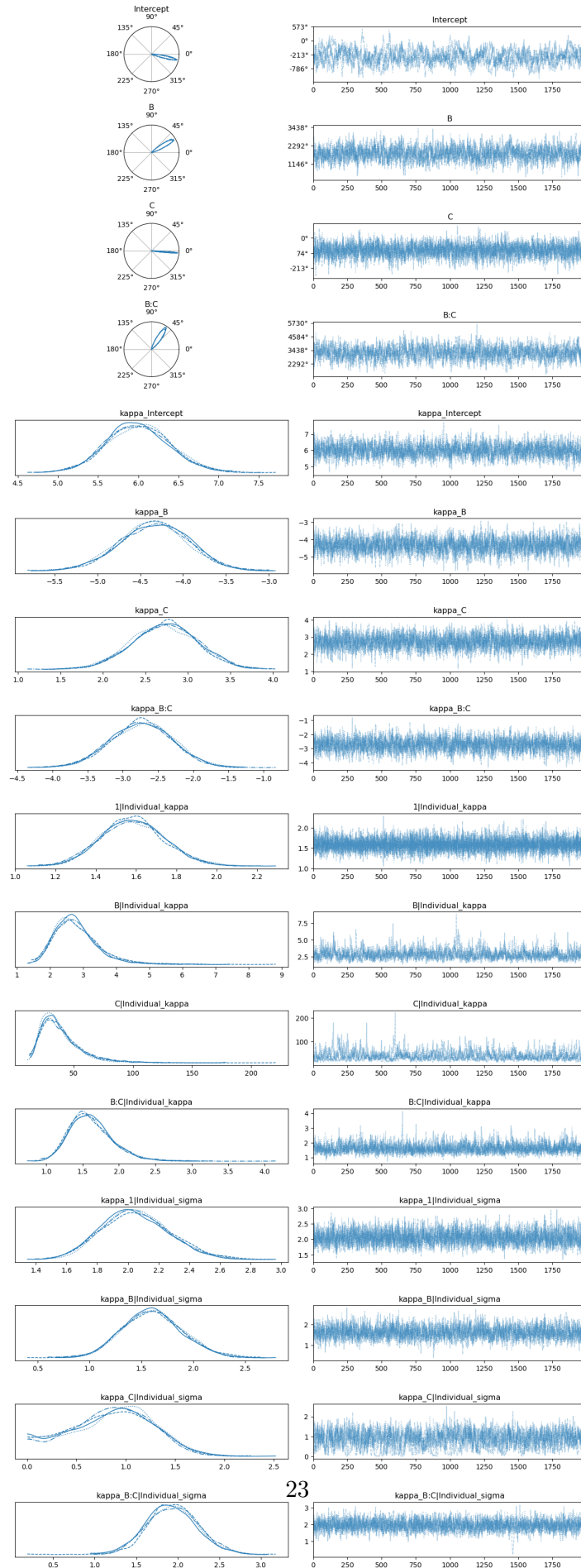
[9]: plot_circular_trace(
      idata = idata_interaction,
      vars=[
        "Intercept",
        "B",
        "C",

```

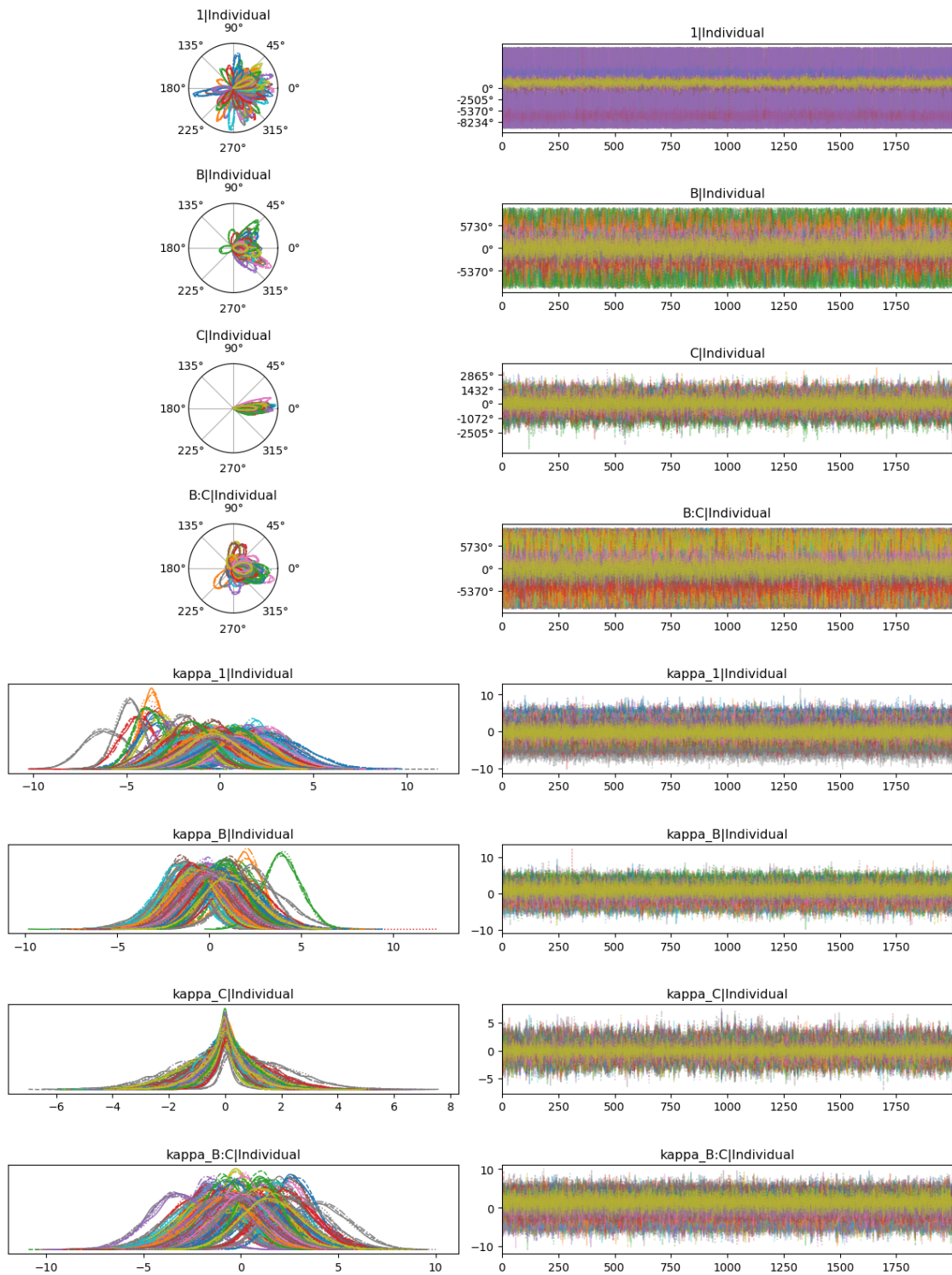
```

        "B:C",
        "kappa_Intercept",
        "kappa_B",
        "kappa_C",
        "kappa_B:C",
        "1|Individual_kappa",
        "B|Individual_kappa",
        "C|Individual_kappa",
        "B:C|Individual_kappa",
        "kappa_1|Individual_sigma",
        "kappa_B|Individual_sigma",
        "kappa_C|Individual_sigma",
        "kappa_B:C|Individual_sigma",
    ],
    cvars = ["Intercept",
            "B",
            "C",
            "B:C",
            ]
)

```



```
[10]: plot_circular_trace(  
      idata = idata_interaction,  
vars=[  
    "1|Individual",  
    "B|Individual",  
    "C|Individual",  
    "B:C|Individual",  
    "kappa_1|Individual",  
    "kappa_B|Individual",  
    "kappa_C|Individual",  
    "kappa_B:C|Individual",  
  ],  
cvars = [  
    "1|Individual",  
    "B|Individual",  
    "C|Individual",  
    "B:C|Individual",  
  ]  
)
```

5 Alternative Hypotheses

We would like to test multiple hypotheses about the data:

0. Are all data explained by individual differences alone? - Individual effect
1. Can all differences be explained by stimulus colour? - UV effect
2. Can all differences be explained by stimulus brightness? - dim effect
3. Does brightness affect accuracy? - $\mu \sim \text{colour}$ - $\kappa \sim \text{colour} \& \text{brightness}$
4. Does colour affect accuracy? - $\mu \sim \text{brightness}$ - $\kappa \sim \text{colour} \& \text{brightness}$
5. Can differences be explained by the combined effects of colour and brightness? - $\mu \sim \text{colour} \& \text{brightness}$ - $\kappa \sim \text{colour} \& \text{brightness}$
6. Is concentration specific to each combinations of colour and brightness? - $\mu \sim \text{colour} \& \text{brightness}$ - $\kappa \sim \text{colour, brightness} \& \text{their interaction}$
7. Is all information about specific combinations of colour and brightness required to explain the data? - $\mu \sim \text{colour, brightness} \& \text{their interaction}$ - $\kappa \sim \text{colour, brightness} \& \text{their interaction}$

5.1 Set up alternative models

Firstly, define the model formulae.

```
[ ]: #model 0
formula_null = bmb.Formula(
    "angle ~ 1 + (1 |Individual)",
    "kappa ~ 1 + (1 |Individual)",
)

#model 1
formula_UV = bmb.Formula(
    "angle ~ C + (1 + C |Individual)",
    "kappa ~ C + (1 + C |Individual)",
)

#model 2
formula_dim = bmb.Formula(
    "angle ~ B + (1 + B |Individual)",
    "kappa ~ B + (1 + B |Individual)",
)

#model 3
formula_UV_dimKappa = bmb.Formula(
    "angle ~ C + (1 + C |Individual)",
    "kappa ~ B + C + (1 + B + C |Individual)",
)

#model 4
formula_dim_UVKappa = bmb.Formula(
    "angle ~ B + (1 + B |Individual)",
    "kappa ~ B + C + (1 + B + C |Individual)",
)

#model 5
```

```

formula_additive = bmb.Formula(
  "angle ~ B + C + (1 + B + C |Individual)",
  "kappa ~ B + C + (1 + B + C |Individual)",
)

#model 6
formula_additive_interactionKappa = bmb.Formula(
  "angle ~ B + C + (1 + B + C |Individual)",
  "kappa ~ B + C + B:C + (1 + B + C + B:C |Individual)",
)

#model 7 already fitted
# "angle ~ B + C + B:C + (1 + B + C + B:C |Individual)",
# "kappa ~ B + C + B:C + (1 + B + C + B:C |Individual)",

```

5.2 Set up priors

By default we use the same priors as for the maximal model although some models require more specific priors to converge.

```

[58]: # =====
# 0. MODEL NULL PRIORS
# =====
priors_null = {
  "Intercept": bmb.Prior("VonMises", mu=0, kappa=1.0), # Low gradient in
↳data, a weak bias to 0°
  "1|Individual": priors_interaction["1|Individual"],
  "kappa": {
    "Intercept": priors_interaction["kappa"]["Intercept"],
    "1|Individual": priors_interaction["kappa"]["1|Individual"]
  }
}

# =====
# 1. MODEL UV PRIORS
# =====
priors_UV = {
  "Intercept": priors_interaction["Intercept"],
  "C": priors_interaction["C"],
  "1|Individual": priors_interaction["1|Individual"],
  "C|Individual": priors_interaction["C|Individual"],
  "kappa": {
    "Intercept": priors_interaction["kappa"]["Intercept"],
    "C": priors_interaction["kappa"]["C"],
    "1|Individual": priors_interaction["kappa"]["1|Individual"],
    "C|Individual": priors_interaction["kappa"]["C|Individual"]
  }
}

```

```

}

# =====
# 2. MODEL DIM PRIORS
# =====
priors_dim = {
  "Intercept": priors_interaction["Intercept"],
  "B": priors_interaction["B"],
  "1|Individual": priors_interaction["1|Individual"],
  "B|Individual": priors_interaction["B|Individual"],
  "kappa": {
    "Intercept": priors_interaction["kappa"]["Intercept"],
    "B": priors_interaction["kappa"]["B"],
    "1|Individual": priors_interaction["kappa"]["1|Individual"],
    "B|Individual": priors_interaction["kappa"]["B|Individual"]
  }
}

# =====
# 3. MODEL UV_DIMKAPPA PRIORS
# =====
priors_UV_dimKappa = {
  "Intercept": priors_interaction["Intercept"],
  "C": priors_interaction["C"],
  "1|Individual": priors_interaction["1|Individual"],
  "C|Individual": priors_interaction["C|Individual"],
  "kappa": {
    "Intercept": bmb.Prior("Normal", mu=3.0, sigma=5.0), #Very wide
    "B": bmb.Prior("Normal", mu=0, sigma=5.0), #Potentially very large
    ↪effect if no change in direction
    "C": bmb.Prior("Normal", mu=0, sigma=5.0), #improved
    "1|Individual": bmb.Prior(
      "Normal", mu=0, sigma=bmb.Prior("HalfStudentT", nu=3, sigma=0.3) #
    ↪Very narrow
    ), #Test
    "B|Individual": bmb.Prior(
      "Normal", mu=0, sigma=bmb.Prior("HalfStudentT", nu=3, sigma=0.3) #
    ↪Very narrow
    ), #Test
    "C|Individual": bmb.Prior(
      "Normal", mu=0, sigma=bmb.Prior("HalfStudentT", nu=3, sigma=0.3) #
    ↪Very narrow
    ), #Test
  }
}

# =====

```

```

# 4. MODEL DIM_UVKAPPA PRIORS
# =====
priors_dim_UVKappa = {
  "Intercept": priors_interaction["Intercept"],
  "B": priors_interaction["B"],
  "1|Individual": priors_interaction["1|Individual"],
  "B|Individual": priors_interaction["B|Individual"],
  "kappa": {
    "Intercept": priors_interaction["kappa"]["Intercept"],
    "B": priors_interaction["kappa"]["B"],
    "C": priors_interaction["kappa"]["C"],
    "1|Individual": priors_interaction["kappa"]["1|Individual"],
    "B|Individual": priors_interaction["kappa"]["B|Individual"],
    "C|Individual": priors_interaction["kappa"]["C|Individual"]
  }
}

# =====
# 5. MODEL ADDITIVE PRIORS
# =====
priors_additive = {
  "Intercept": priors_interaction["Intercept"],
  "B": priors_interaction["B"],
  "C": priors_interaction["C"],
  "1|Individual": priors_interaction["1|Individual"],
  "B|Individual": priors_interaction["B|Individual"],
  "C|Individual": priors_interaction["C|Individual"],
  "kappa": {
    "Intercept": priors_interaction["kappa"]["Intercept"],
    "B": priors_interaction["kappa"]["B"],
    "C": priors_interaction["kappa"]["C"],
    "1|Individual": priors_interaction["kappa"]["1|Individual"],
    "B|Individual": priors_interaction["kappa"]["B|Individual"],
    "C|Individual": priors_interaction["kappa"]["C|Individual"]
  }
}

# =====
# 6. MODEL ADDITIVE_INTERACTIONKAPPA PRIORS
# =====
priors_additive_interactionKappa = {
  "Intercept": priors_interaction["Intercept"],
  "B": priors_interaction["B"],
  "C": priors_interaction["C"],
  "1|Individual": priors_interaction["1|Individual"],
  "B|Individual": priors_interaction["B|Individual"],
  "C|Individual": priors_interaction["C|Individual"],

```

```

    "kappa": {
      "Intercept": priors_interaction["kappa"]["Intercept"],
      "B": priors_interaction["kappa"]["B"],
      "C": priors_interaction["kappa"]["C"],
      "B:C": priors_interaction["kappa"]["B:C"],
      "1|Individual": priors_interaction["kappa"]["1|Individual"],
      "B|Individual": priors_interaction["kappa"]["B|Individual"],
      "C|Individual": priors_interaction["kappa"]["C|Individual"],
      "B:C|Individual": priors_interaction["kappa"]["B:C|Individual"]
    }
  }
}

```

5.3 Prepare all models

Pair each model with its specific formula and priors, but the same data and variables.

```

[59]: #model 0
model_null = bmb.Model(
  formula_null,
  df,
  family="vonmises",
  priors=priors_null,
  link={"mu": LINKS["shifted_modulo"], "kappa": LINKS["softplus"]},
  center_predictors=False,
  dropna=True,
)

#model 1
model_UV = bmb.Model(
  formula_UV,
  df,
  family="vonmises",
  priors=priors_UV,
  link={"mu": LINKS["shifted_modulo"], "kappa": LINKS["softplus"]},
  center_predictors=False,
  dropna=True,
)

#model 2
model_dim = bmb.Model(
  formula_dim,
  df,
  family="vonmises",
  priors=priors_dim,
  link={"mu": LINKS["shifted_modulo"], "kappa": LINKS["softplus"]},
  center_predictors=False,
)

```

```

    dropna=True,
)

#model 3
model_UV_dimKappa = bmb.Model(
    formula_UV_dimKappa,
    df,
    family="vonmises",
    priors=priors_UV_dimKappa,
    link={"mu": LINKS["shifted_modulo"], "kappa": LINKS["softplus"]},
    center_predictors=False,
    dropna=True,
)

#model 4
model_dim_UVKappa = bmb.Model(
    formula_dim_UVKappa,
    df,
    family="vonmises",
    priors=priors_dim_UVKappa,
    link={"mu": LINKS["shifted_modulo"], "kappa": LINKS["softplus"]},
    center_predictors=False,
    dropna=True,
)

#model 5
model_additive = bmb.Model(
    formula_additive,
    df,
    family="vonmises",
    priors=priors_additive,
    link={"mu": LINKS["shifted_modulo"], "kappa": LINKS["softplus"]},
    center_predictors=False,
    dropna=True,
)

#model 6
model_additive_interactionKappa = bmb.Model(
    formula_additive_interactionKappa,
    df,
    family="vonmises",
    priors=priors_additive_interactionKappa,
    link={"mu": LINKS["shifted_modulo"], "kappa": LINKS["softplus"]},
    center_predictors=False,
    dropna=True,
)

```

5.4 Run Null model (0)

Null (Intercept and Individual-level random intercept only)

```
[16]: # Fit the model.
idata_null = model_null.fit(
    draws=2000,          # longer run for mu convergence
    tune=1000,
    progressbar="split+stats",
    nuts_sampler="nutpie", # build and route via nutpie automatically
    target_accept=0.98    # matching the high adapt delta used in model_UV
)

# check convergence heuristics for fixed/population effects
az.summary(
    idata_null,
    var_names=[
        "Intercept",
        "kappa_Intercept",
    ],
    circ_var_names=[
        "Intercept",
    ]
)
```

```
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/pymc/sampling/mcmc.py:303: UserWarning: `var_names` are currently
ignored by the nutpie sampler
```

```
warnings.warn(
```

```
<IPython.core.display.HTML object>
```

```
<nutpie.sample._BackgroundSampler at 0x1ad48ede0>
```

```
[16]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	\
Intercept	-0.018	0.108	-0.225	0.177	0.006	0.005	200.0	
kappa_Intercept	1.055	0.110	0.849	1.266	0.004	0.003	913.0	

	ess_tail	r_hat
Intercept	337.0	1.01
kappa_Intercept	1885.0	1.01

```
[17]: # check convergence heuristics on group-specific/random effects
az.summary(
    idata_null,
    var_names=[
        "1|Individual_kappa",
        "kappa_1|Individual_sigma",
    ],
)
```



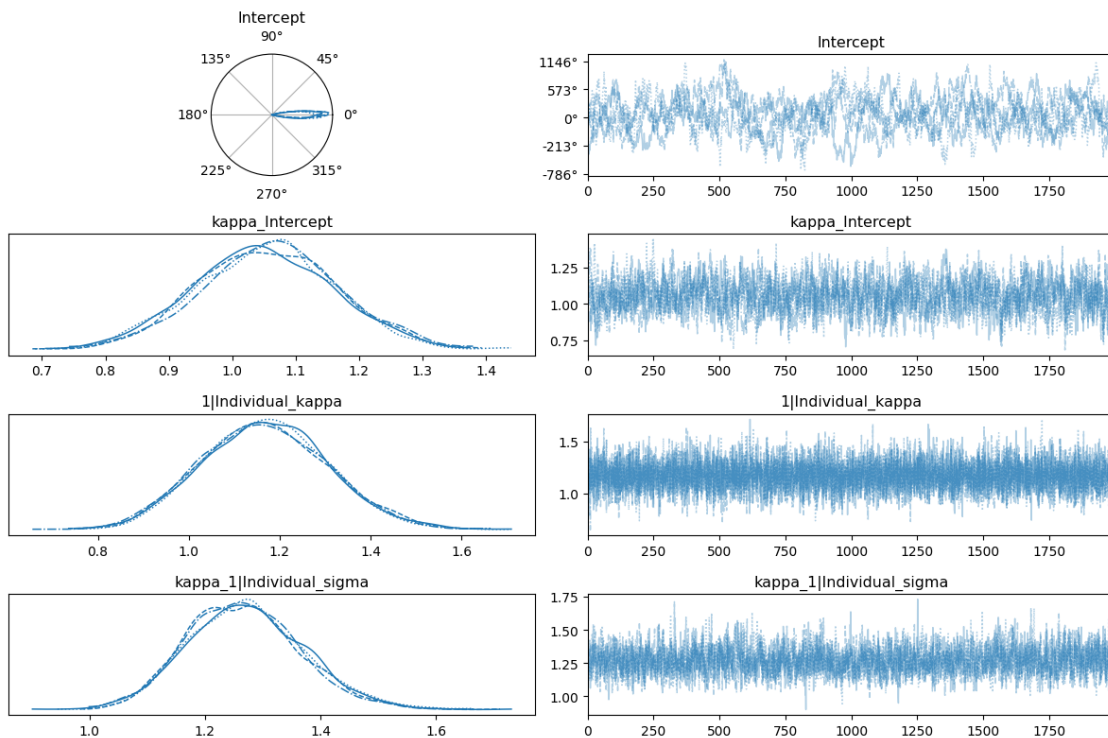
```
[17]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	\
1 Individual_kappa	1.169	0.138	0.912	1.426	0.001	0.001	
kappa_1 Individual_sigma	1.269	0.099	1.092	1.461	0.002	0.002	

	ess_bulk	ess_tail	r_hat
1 Individual_kappa	8737.0	5466.0	1.0
kappa_1 Individual_sigma	2116.0	3663.0	1.0

```
[18]: # Plot traces for all null model parameters
```

```
plot_circular_trace(
  idata=idata_null,
  vars=[
    "Intercept",
    "kappa_Intercept",
    "1|Individual_kappa",
    "kappa_1|Individual_sigma",
  ],
  cvars=[
    "Intercept",
  ]
)
```



5.5 Run colour model (1)

UV (C only)

```
[19]: import numpy as np
import pandas as pd
import bambi as bmb
import arviz as az
import matplotlib.pyplot as plt

# Fit the model.
idata_UV = model_UV.fit(draws= 2000, #longer run for mu convergence
                        tune = 1000,
                        progressbar="split+stats",
                        nuts_sampler="nutpie", #build and route via nutpie
                        ↪automatically
                        target_accept= 0.98 # Needs sampling with small steps or
                        ↪generates some divergences 0.95 works
                        ) # be prepared for this to take 90 minutes with high adapt delta
#check convergence heuristics
az.summary(
    idata_UV,
    var_names=["Intercept", #slightly right shifted, should converge to around
    ↪0.26
               "C",
               "kappa_Intercept",
               "kappa_C",
               ],
    circ_var_names=["Intercept",
                    "C",
                    ]
)
```

```
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/pymc/sampling/mcmc.py:303: UserWarning: `var_names` are currently
ignored by the nutpie sampler
  warnings.warn(
```

```
<IPython.core.display.HTML object>
```

```
<nutpie.sample._BackgroundSampler at 0x19a816ae0>
```

```
[19]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	\
Intercept	0.042	0.121	-0.194	0.258	0.005	0.003	794.0	
C	-0.033	0.061	-0.143	0.088	0.001	0.001	3924.0	
kappa_Intercept	2.432	0.216	2.019	2.832	0.004	0.003	2981.0	
kappa_C	-1.573	0.234	-2.020	-1.145	0.004	0.003	3054.0	

ess_tail r_hat

Intercept	1449.0	1.0
C	4514.0	1.0
kappa_Intercept	4517.0	1.0
kappa_C	4460.0	1.0

[20]: *#check convergence heuristics on random effects*

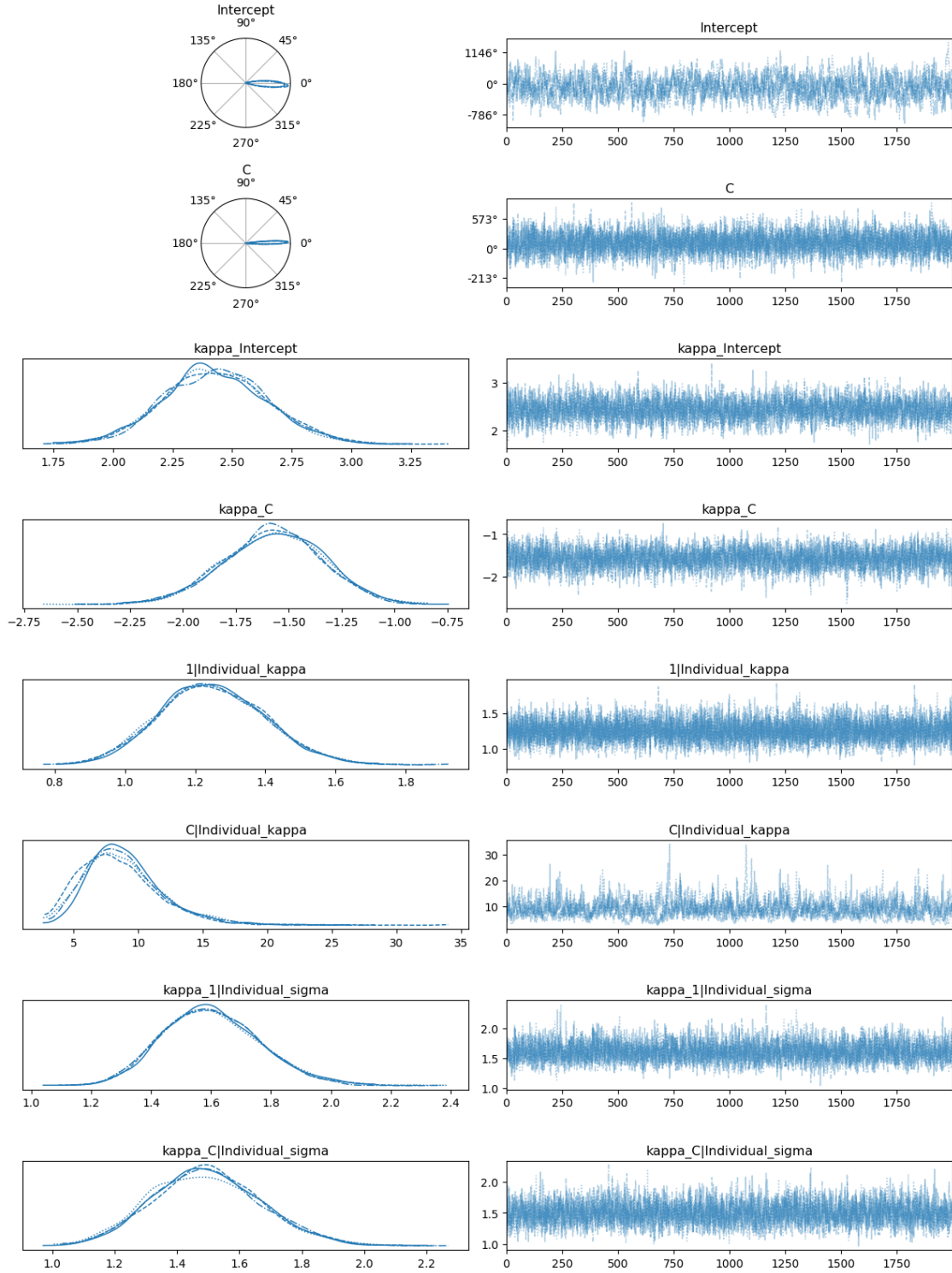
```
az.summary(
    idata_UV,
    var_names=[
        "1|Individual_kappa",
        "C|Individual_kappa",
        "kappa_1|Individual_sigma",
        "kappa_C|Individual_sigma",
    ],
)
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	\
1 Individual_kappa	1.250	0.149	0.969	1.527	0.002	0.001	
C Individual_kappa	8.850	3.138	3.419	14.372	0.127	0.090	
kappa_1 Individual_sigma	1.603	0.162	1.299	1.899	0.003	0.002	
kappa_C Individual_sigma	1.497	0.174	1.162	1.818	0.003	0.002	

	ess_bulk	ess_tail	r_hat
1 Individual_kappa	7602.0	6769.0	1.00
C Individual_kappa	440.0	282.0	1.01
kappa_1 Individual_sigma	3196.0	4861.0	1.00
kappa_C Individual_sigma	2951.0	5091.0	1.00

[21]: `plot_circular_trace(`

```
    idata = idata_UV,
    vars=[
        "Intercept",
        "C",
        "kappa_Intercept",
        "kappa_C",
        "1|Individual_kappa",
        "C|Individual_kappa",
        "kappa_1|Individual_sigma",
        "kappa_C|Individual_sigma",
    ],
    cvars = ["Intercept",
             "C",
             ]
)
```



5.6 Run brightness model (2)

dim (B only)

```
[22]: # Fit the model.
idata_dim = model_dim.fit(draws= 2000, #longer run for mu convergence
                           tune = 1000,
                           progressbar="split+stats",
                           nuts_sampler="nutpie", #build and route via nutpie
                           ↪automatically
                           target_accept= 0.98 # Needs sampling with small steps or
                           ↪generates some divergences 0.95 works
                           ) # be prepared for this to take 90 minutes with high adapt delta
#check convergence heuristics
az.summary(
    idata_dim,
    var_names=["Intercept", #slightly right shifted, should converge to around
    ↪0.26
               "B",
               "kappa_Intercept",
               "kappa_B",
               ],
    circ_var_names=["Intercept",
                    "B",
                    ]
)
```

```
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/pymc/sampling/mcmc.py:303: UserWarning: `var_names` are currently
ignored by the nutpie sampler
  warnings.warn(
```

```
<IPython.core.display.HTML object>
```

```
<nutpie.sample._BackgroundSampler at 0x19932e840>
```

```
[22]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	\
Intercept	0.254	0.080	0.103	0.405	0.006	0.003	276.0	
B	-1.417	0.127	-1.660	-1.188	0.003	0.003	850.0	
kappa_Intercept	7.887	0.285	7.365	8.447	0.005	0.003	3389.0	
kappa_B	-6.386	0.298	-6.928	-5.826	0.005	0.004	3520.0	

	ess_tail	r_hat
Intercept	682.0	1.02
B	1491.0	1.00
kappa_Intercept	4717.0	1.00
kappa_B	4506.0	1.00

```
[23]: #check convergence heuristics on random effects
az.summary(
    idata_dim,
    var_names=[
```

```

        "1|Individual_kappa",
        "B|Individual_kappa",
        "kappa_1|Individual_sigma",
        "kappa_"
        "B|Individual_sigma",
    ],
)

```

```

[23]:
      mean      sd  hdi_3%  hdi_97%  mcse_mean  mcse_sd  \
1|Individual_kappa    1.564  0.150    1.276    1.838      0.001   0.001
B|Individual_kappa    1.139  0.148    0.862    1.412      0.002   0.001
kappa_1|Individual_sigma  2.245  0.209    1.845    2.626      0.004   0.003
kappa_B|Individual_sigma  2.390  0.261    1.902    2.880      0.005   0.004

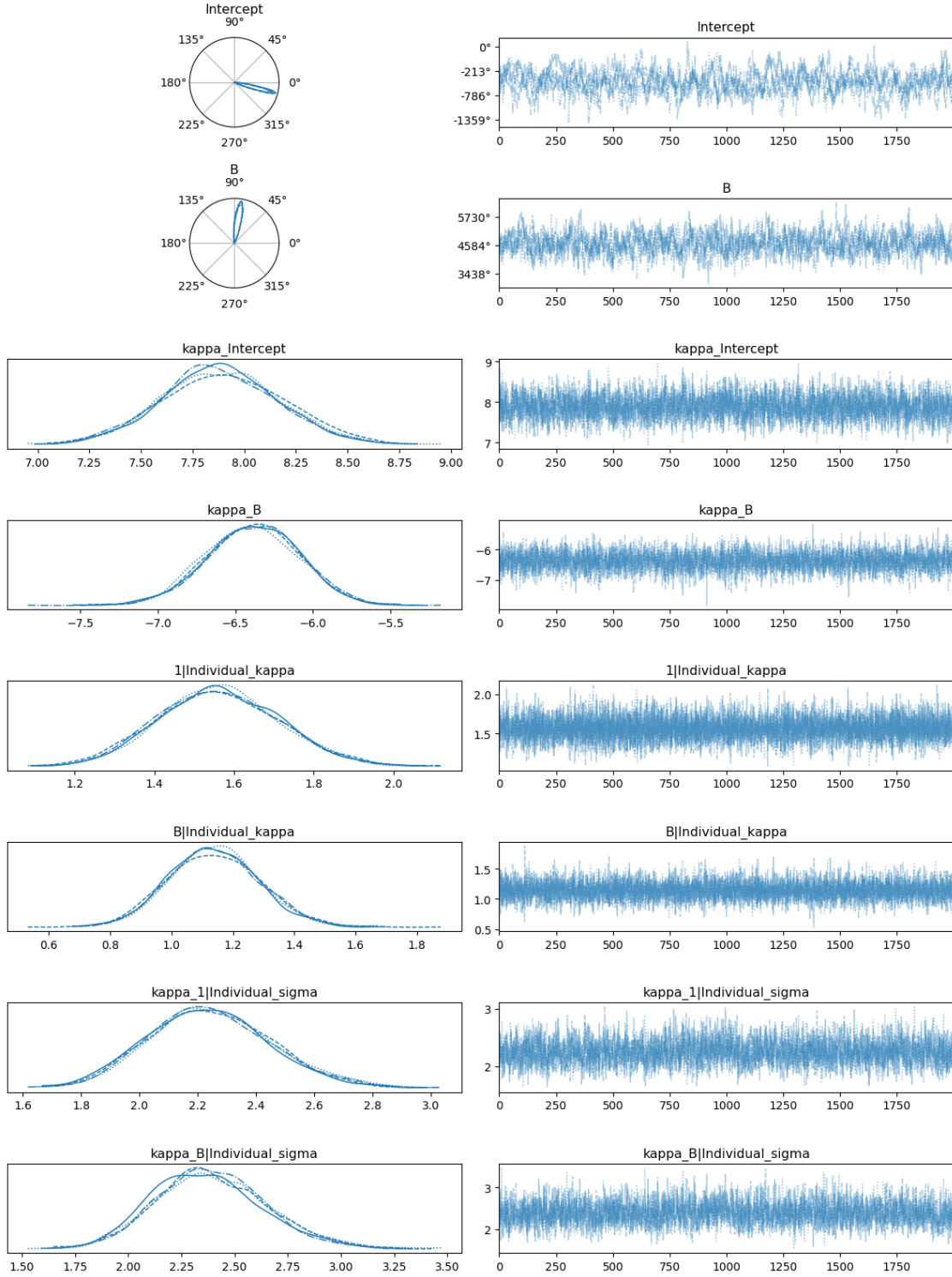
      ess_bulk  ess_tail  r_hat
1|Individual_kappa    11437.0    4901.0    1.0
B|Individual_kappa     8180.0    6038.0    1.0
kappa_1|Individual_sigma  2431.0    4146.0    1.0
kappa_B|Individual_sigma  2452.0    4201.0    1.0

```

```

[24]: plot_circular_trace(
        idata = idata_dim,
        vars=[
            "Intercept",
            "B",
            "kappa_Intercept",
            "kappa_B",
            "1|Individual_kappa",
            "B|Individual_kappa",
            "kappa_1|Individual_sigma",
            "kappa_B|Individual_sigma",
        ],
        cvars = ["Intercept",
            "B",
        ]
    )

```



5.7 Run colour & kappa-brightness model (3)

UV_dimKappa (angle $\sim$ C, kappa $\sim$ B + C)

```
[60]: # Fit the model.
idata_UV_dimKappa = model_UV_dimKappa.fit(draws= 2000,#longer run for mu
↳convergence

tune=1000,
progressbar="split+stats",
nuts_sampler="nutpie", #build and
↳route via nutpie automatically

target_accept=0.98)

# Fixed effects summary
az.summary(idata_UV_dimKappa,
var_names=["Intercept",
"C",
"kappa_Intercept",
"kappa_B",
"kappa_C"],
circ_var_names=["Intercept",
"C"])
```

python(57019) MallocStackLogging: can't turn off malloc stack logging because it was not enabled.

/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-

packages/pymc/sampling/mcmc.py:303: UserWarning: `var\_names` are currently ignored by the nutpie sampler

warnings.warn(

<IPython.core.display.HTML object>

<nutpie.sample._BackgroundSampler at 0x1b68c10a0>

```
[60]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	\
Intercept	0.161	0.092	-0.015	0.327	0.002	0.003	501.0	
C	0.061	0.035	-0.004	0.128	0.002	0.002	187.0	
kappa_Intercept	10.168	0.672	8.901	11.408	0.013	0.009	2724.0	
kappa_B	-16.219	1.198	-18.543	-14.036	0.027	0.019	1983.0	
kappa_C	-0.716	0.621	-1.902	0.435	0.012	0.009	2651.0	

	ess_tail	r_hat
Intercept	1022.0	1.01
C	4309.0	1.02
kappa_Intercept	4426.0	1.00
kappa_B	3001.0	1.00
kappa_C	3965.0	1.00

```
[61]: # Random effects summary
az.summary(idata_UV_dimKappa,
var_names=["1|Individual_kappa",
"C|Individual_kappa",
```



```

    "kappa_1|Individual_sigma",
    "kappa_B|Individual_sigma",
    "kappa_C|Individual_sigma"]])

```

```

[61]:

```

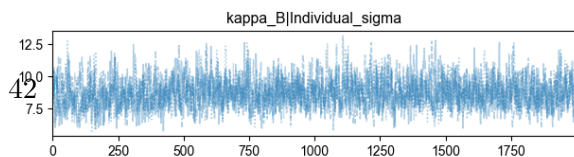
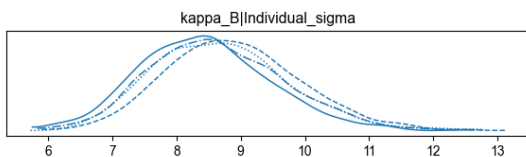
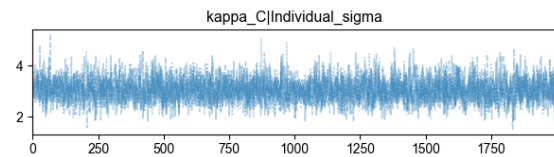
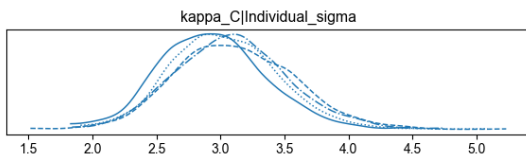
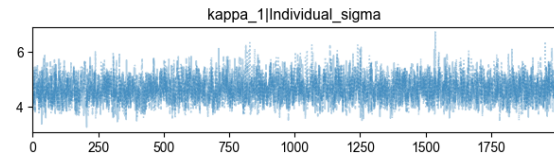
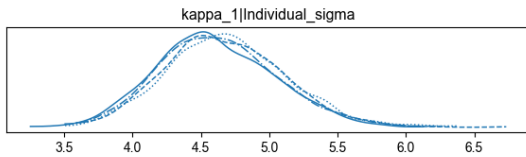
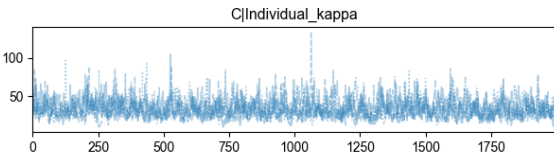
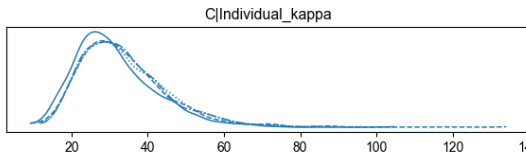
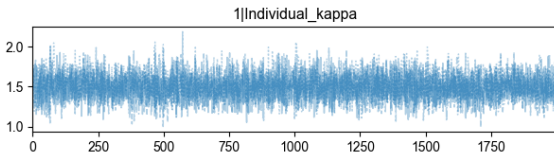
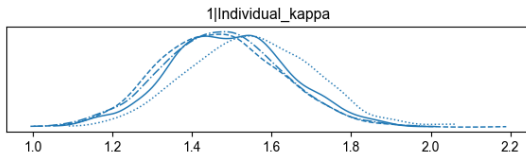
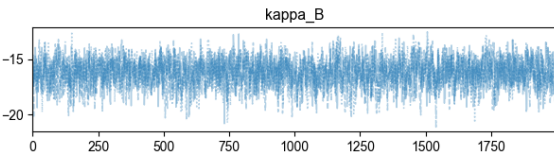
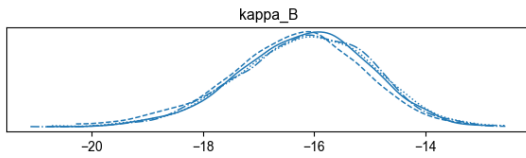
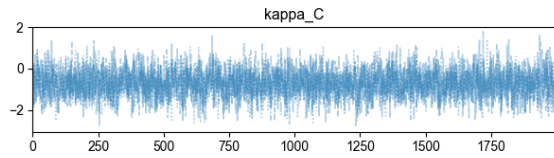
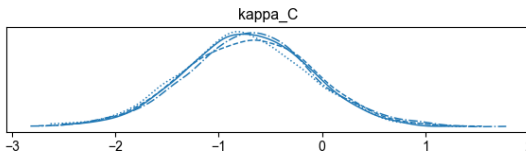
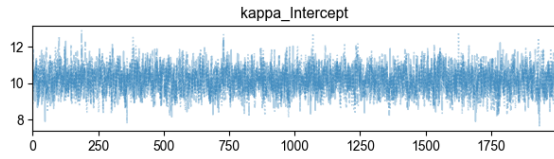
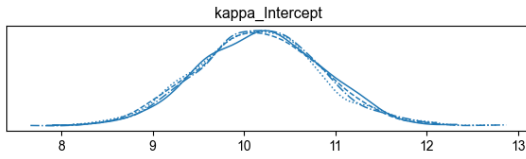
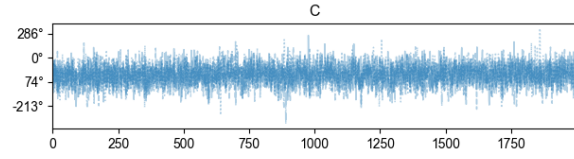
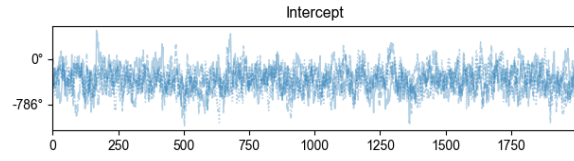
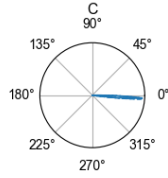
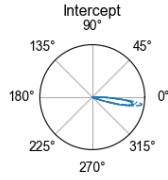
	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	\
1 Individual_kappa	1.497	0.155	1.218	1.792	0.014	0.010	
C Individual_kappa	33.083	11.359	14.692	54.160	0.388	0.274	
kappa_1 Individual_sigma	4.650	0.434	3.823	5.430	0.011	0.008	
kappa_B Individual_sigma	8.681	1.085	6.773	10.768	0.053	0.037	
kappa_C Individual_sigma	3.041	0.438	2.245	3.877	0.020	0.014	

	ess_bulk	ess_tail	r_hat
1 Individual_kappa	123.0	3967.0	1.03
C Individual_kappa	784.0	1714.0	1.01
kappa_1 Individual_sigma	1350.0	3816.0	1.01
kappa_B Individual_sigma	363.0	1883.0	1.02
kappa_C Individual_sigma	460.0	4237.0	1.02

```

[62]: plot_circular_trace(
    idata = idata_UV_dimKappa,
    vars=[
        "Intercept",
        "C",
        "kappa_Intercept",
        "kappa_C",
        "kappa_B",
        "1|Individual_kappa",
        "C|Individual_kappa",
        "kappa_1|Individual_sigma",
        "kappa_C|Individual_sigma",
        "kappa_B|Individual_sigma",
    ],
    cvars = ["Intercept",
            "C",
            ]
)

```



5.8 Run brightness & kappa-colour model (4)

Dim_UVKappa (angle $\sim$ B, kappa $\sim$ B + C)

```
[28]: # Fit the model.
idata_dim_UVKappa = model_dim_UVKappa.fit(draws= 2000, #longer run for mu
↳convergence

                                         tune=1000,
                                         progressbar="split+stats",
                                         nuts_sampler="nutpie", #build and
↳route via nutpie automatically

                                         target_accept=0.98)

# Fixed effects summary
az.summary(idata_dim_UVKappa,
  var_names=["Intercept",
            "B",
            "kappa_Intercept",
            "kappa_B",
            "kappa_C"],
  circ_var_names=["Intercept",
                 "B"])
```

```
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/pymc/sampling/mcmc.py:303: UserWarning: `var_names` are currently
ignored by the nutpie sampler
  warnings.warn(
```

```
<IPython.core.display.HTML object>
```

```
<nutpie.sample._BackgroundSampler at 0x19f118620>
```

```
[28]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	\
Intercept	0.255	0.079	0.111	0.400	0.004	0.004	183.0	
B	-1.470	0.135	-1.727	-1.217	0.007	0.005	348.0	
kappa_Intercept	7.237	0.404	6.450	7.969	0.015	0.010	760.0	
kappa_B	-6.749	0.310	-7.327	-6.144	0.006	0.005	2281.0	
kappa_C	1.224	0.359	0.535	1.879	0.013	0.009	823.0	

	ess_tail	r_hat
Intercept	414.0	1.02
B	603.0	1.02
kappa_Intercept	1340.0	1.01
kappa_B	3932.0	1.00
kappa_C	2935.0	1.01

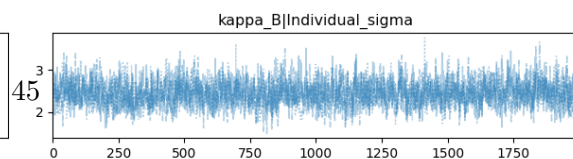
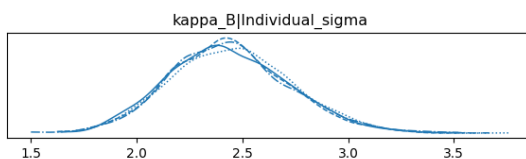
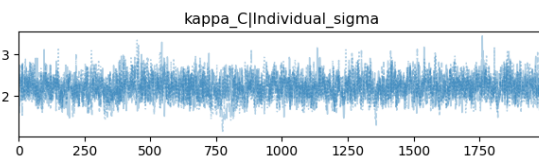
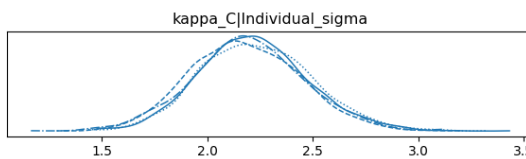
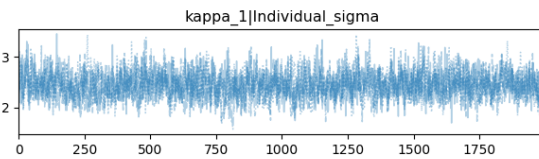
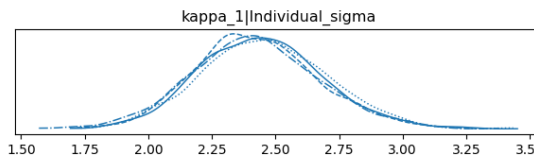
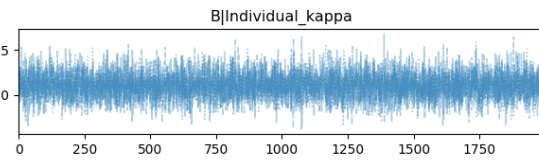
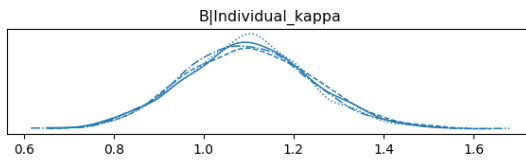
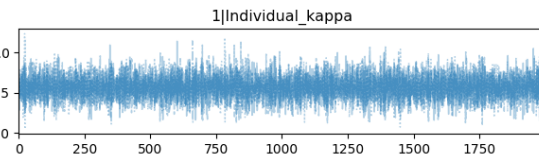
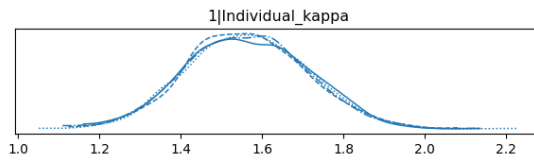
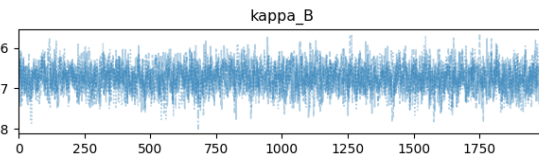
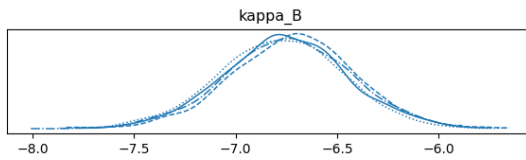
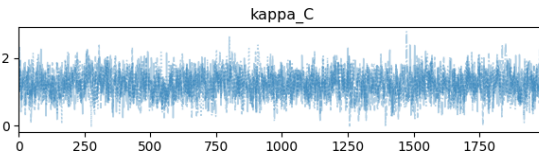
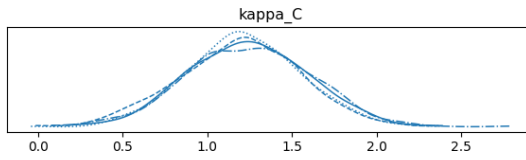
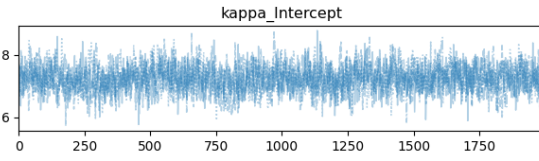
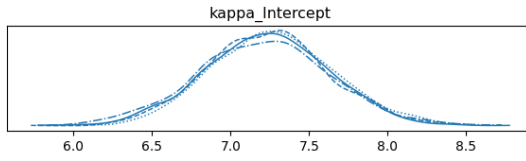
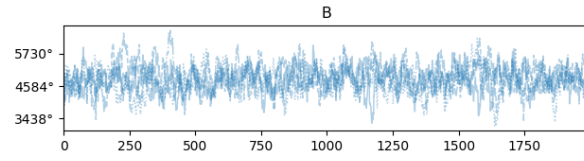
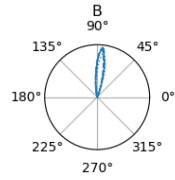
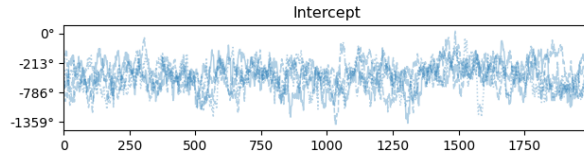
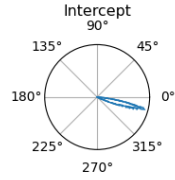
```
[29]: # Random effects summary
az.summary(idata_dim_UVKappa,
           var_names=["1|Individual_kappa",
                      "B|Individual_kappa",
                      "kappa_1|Individual_sigma",
                      "kappa_B|Individual_sigma",
                      "kappa_C|Individual_sigma"])
```

```
[29]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	\
1 Individual_kappa	1.563	0.149	1.302	1.854	0.001	0.001	
B Individual_kappa	1.099	0.142	0.830	1.365	0.002	0.001	
kappa_1 Individual_sigma	2.438	0.249	1.978	2.921	0.008	0.006	
kappa_B Individual_sigma	2.441	0.288	1.920	2.994	0.008	0.006	
kappa_C Individual_sigma	2.208	0.266	1.724	2.720	0.008	0.006	

	ess_bulk	ess_tail	r_hat
1 Individual_kappa	17262.0	5372.0	1.0
B Individual_kappa	7183.0	5615.0	1.0
kappa_1 Individual_sigma	971.0	1695.0	1.0
kappa_B Individual_sigma	1318.0	2248.0	1.0
kappa_C Individual_sigma	1041.0	1281.0	1.0

```
[30]: plot_circular_trace(
           idata = idata_dim_UVKappa,
           vars=[
               "Intercept",
               "B",
               "kappa_Intercept",
               "kappa_C",
               "kappa_B",
               "1|Individual_kappa",
               "B|Individual_kappa",
               "kappa_1|Individual_sigma",
               "kappa_C|Individual_sigma",
               "kappa_B|Individual_sigma",
           ],
           cvars = ["Intercept",
                   "B",
                   ]
       )
```



5.9 Run additive model (5)

Additive (B + C on both)

```
[31]: # Fit the model.
idata_additive = model_additive.fit(draws= 2000, #longer run for mu convergence
                                     tune=1000,
                                     progressbar="split+stats",
                                     nuts_sampler="nutpie", #build and route
                                     ↪via nutpie automatically
                                     target_accept=0.98)

# Fixed effects summary
az.summary(idata_additive,
           var_names=["Intercept",
                      "B",
                      "C",
                      "kappa_Intercept",
                      "kappa_B",
                      "kappa_C"],
           circ_var_names=["Intercept",
                          "B",
                          "C"])
```

```
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/pymc/sampling/mcmc.py:303: UserWarning: `var_names` are currently
ignored by the nutpie sampler
  warnings.warn(
```

```
<IPython.core.display.HTML object>
```

```
<nutpie.sample._BackgroundSampler at 0x1aad8f770>
```

```
[31]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	\
Intercept	0.257	0.090	0.088	0.426	0.002	0.002	656.0	
B	-1.485	0.128	-1.722	-1.239	0.006	0.003	803.0	
C	-0.001	0.038	-0.074	0.069	0.000	0.000	3901.0	
kappa_Intercept	7.374	0.402	6.626	8.146	0.008	0.006	2333.0	
kappa_B	-6.773	0.314	-7.345	-6.159	0.005	0.003	4357.0	
kappa_C	1.136	0.347	0.485	1.784	0.007	0.005	2229.0	

	ess_tail	r_hat
Intercept	1444.0	1.00
B	1386.0	1.01
C	4972.0	1.00
kappa_Intercept	3637.0	1.00

kappa_B	5355.0	1.00
kappa_C	3417.0	1.00

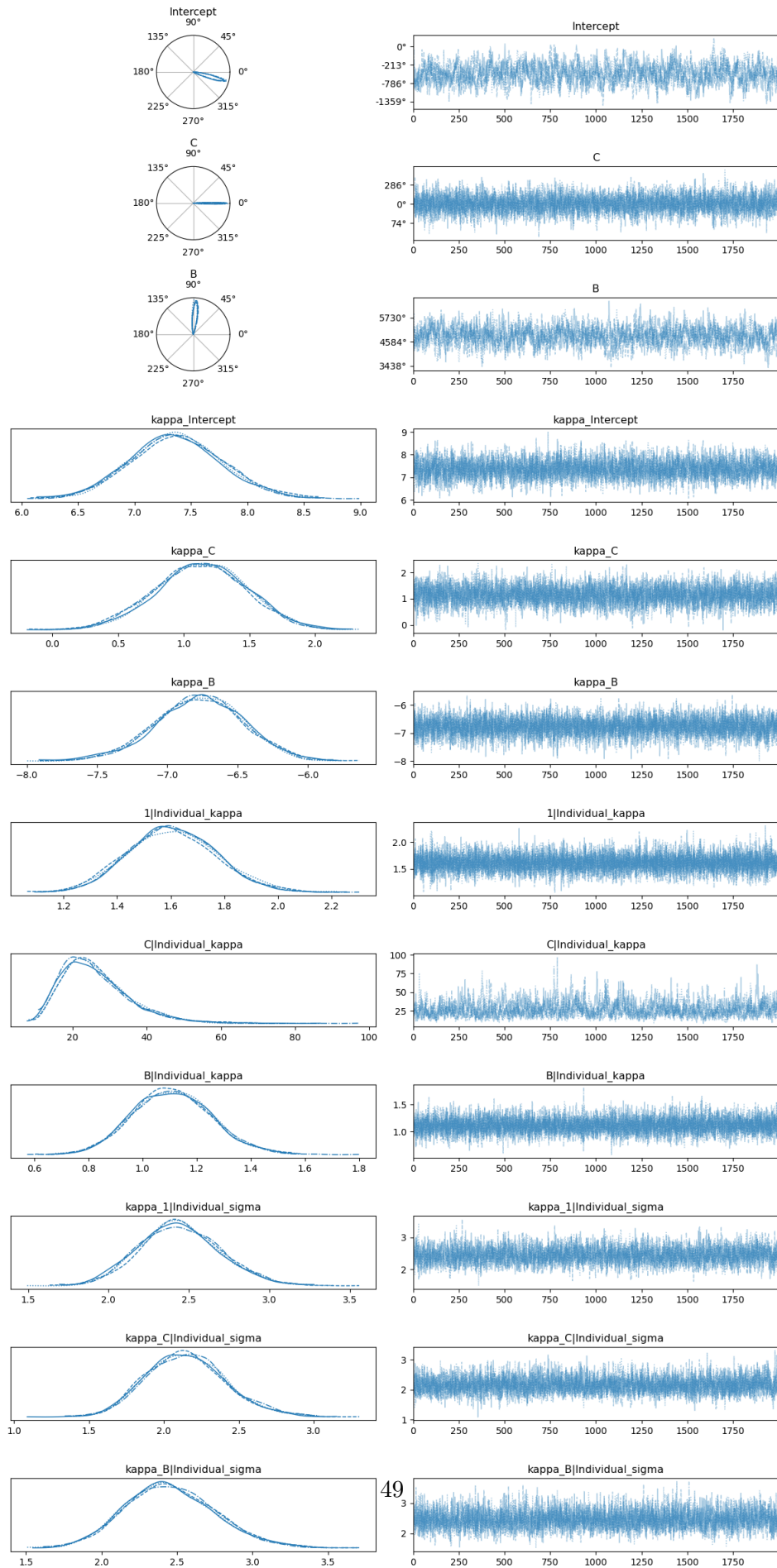
```
[32]: # Random effects summary
az.summary(idata_additive,
           var_names=["1|Individual_kappa",
                      "B|Individual_kappa",
                      "C|Individual_kappa",
                      "kappa_1|Individual_sigma",
                      "kappa_B|Individual_sigma",
                      "kappa_C|Individual_sigma"])
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	\
1 Individual_kappa	1.607	0.163	1.301	1.912	0.002	0.001	
B Individual_kappa	1.110	0.145	0.830	1.379	0.002	0.001	
C Individual_kappa	26.345	9.434	12.089	44.558	0.362	0.256	
kappa_1 Individual_sigma	2.443	0.251	1.975	2.916	0.005	0.004	
kappa_B Individual_sigma	2.458	0.300	1.895	3.020	0.006	0.004	
kappa_C Individual_sigma	2.151	0.267	1.678	2.678	0.005	0.003	

	ess_bulk	ess_tail	r_hat
1 Individual_kappa	8565.0	5382.0	1.0
B Individual_kappa	5804.0	5614.0	1.0
C Individual_kappa	692.0	1452.0	1.0
kappa_1 Individual_sigma	2433.0	4491.0	1.0
kappa_B Individual_sigma	2271.0	3585.0	1.0
kappa_C Individual_sigma	2952.0	4167.0	1.0

```
[33]: plot_circular_trace(
        idata = idata_additive,
        vars=[
            "Intercept",
            "C",
            "B",
            "kappa_Intercept",
            "kappa_C",
            "kappa_B",
            "1|Individual_kappa",
            "C|Individual_kappa",
            "B|Individual_kappa",
            "kappa_1|Individual_sigma",
            "kappa_C|Individual_sigma",
            "kappa_B|Individual_sigma",
        ],
        cvars = ["Intercept",
                 "C",
                 "B",
```

)]



5.10 Run additive & kappa-interaction model (6)

μ additive (B + C), κ interaction (B * C)

```
[34]: # Fit the model.
idata_additive_interactionKappa = model_additive_interactionKappa.fit(draws=
    ↪2000, #longer run for mu convergence
                                tune=1000,
                                progressbar="split+stats",
                                nuts_sampler="nutpie", #build and route
    ↪via nutpie automatically
                                target_accept=0.98)

# Fixed effects summary
az.summary(idata_additive_interactionKappa,
            var_names=["Intercept",
                        "B",
                        "C",
                        "kappa_Intercept",
                        "kappa_B",
                        "kappa_C",
                        "kappa_B:C",
                        ],
            circ_var_names=["Intercept",
                            "B",
                            "C"])
```

```
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/pymc/sampling/mcmc.py:303: UserWarning: `var_names` are currently
ignored by the nutpie sampler
  warnings.warn(
```

```
<IPython.core.display.HTML object>
```

```
<nutpie.sample._BackgroundSampler at 0x1b07648f0>
```

```
[34]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	\
Intercept	0.265	0.085	0.106	0.426	0.003	0.003	361.0	
B	-1.408	0.125	-1.642	-1.177	0.004	0.004	439.0	
C	-0.003	0.038	-0.075	0.068	0.000	0.001	2361.0	
kappa_Intercept	6.135	0.406	5.366	6.883	0.010	0.007	1712.0	
kappa_B	-4.592	0.426	-5.381	-3.797	0.014	0.010	923.0	
kappa_C	2.698	0.397	1.962	3.427	0.009	0.006	1933.0	
kappa_B:C	-2.753	0.483	-3.675	-1.851	0.016	0.011	975.0	

```

    ess_tail  r_hat
```

Intercept	581.0	1.00
B	1081.0	1.00
C	3543.0	1.00
kappa_Intercept	2944.0	1.00
kappa_B	2178.0	1.01
kappa_C	3549.0	1.00
kappa_B:C	2162.0	1.01

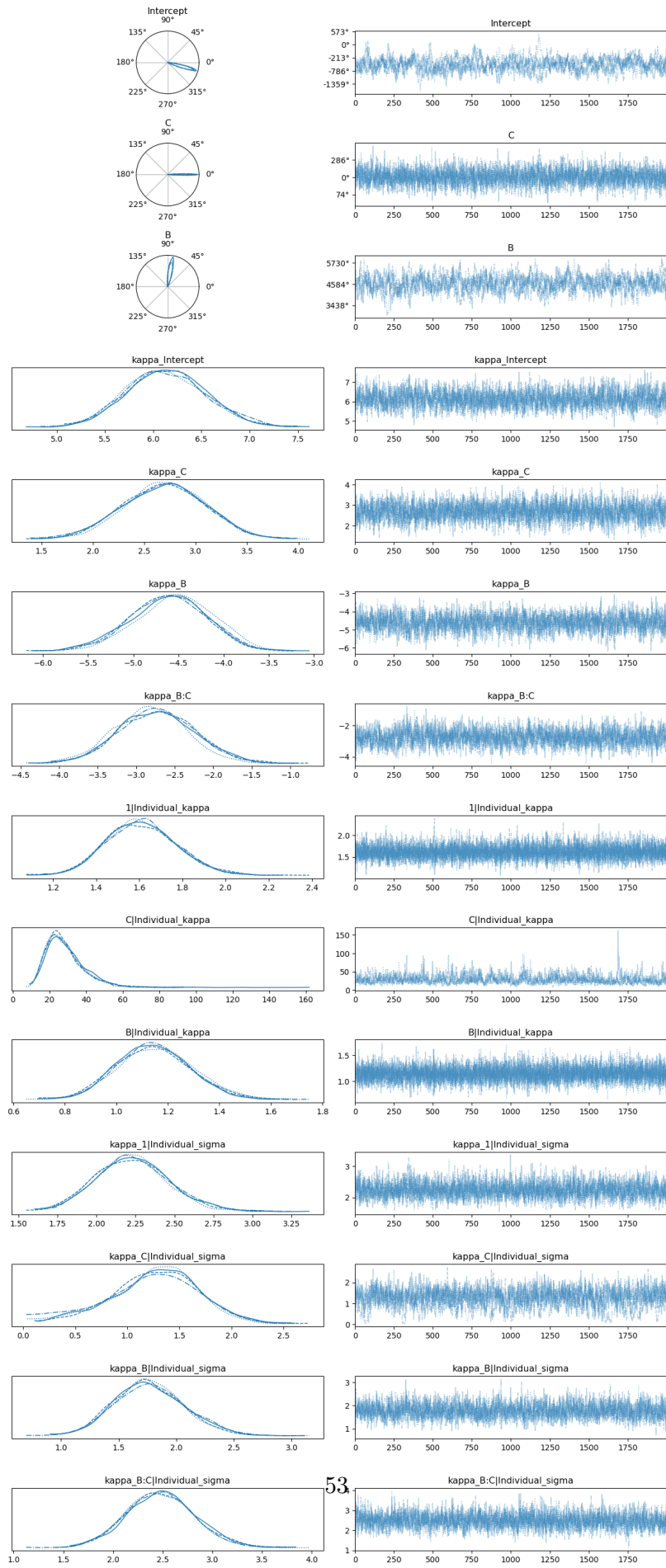
```
[35]: # Random effects summary
az.summary(idata_additive_interactionKappa,
           var_names=["1|Individual_kappa",
                      "B|Individual_kappa",
                      "C|Individual_kappa",
                      "kappa_1|Individual_sigma",
                      "kappa_B|Individual_sigma",
                      "kappa_C|Individual_sigma",
                      "kappa_B:C|Individual_sigma",
                      ])
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	\
1 Individual_kappa	1.611	0.161	1.305	1.906	0.002	
B Individual_kappa	1.147	0.144	0.887	1.419	0.002	
C Individual_kappa	28.561	10.794	11.926	47.887	0.427	
kappa_1 Individual_sigma	2.230	0.236	1.793	2.674	0.005	
kappa_B Individual_sigma	1.769	0.302	1.210	2.343	0.007	
kappa_C Individual_sigma	1.253	0.429	0.337	1.993	0.018	
kappa_B:C Individual_sigma	2.478	0.345	1.834	3.139	0.008	

	mcse_sd	ess_bulk	ess_tail	r_hat
1 Individual_kappa	0.001	11100.0	5575.0	1.00
B Individual_kappa	0.001	5640.0	6370.0	1.00
C Individual_kappa	0.302	570.0	1337.0	1.01
kappa_1 Individual_sigma	0.004	2164.0	3497.0	1.00
kappa_B Individual_sigma	0.005	1989.0	3842.0	1.00
kappa_C Individual_sigma	0.013	616.0	537.0	1.00
kappa_B:C Individual_sigma	0.006	1791.0	3091.0	1.00

```
[36]: plot_circular_trace(
           idata = idata_additive_interactionKappa,
           vars=[
               "Intercept",
               "C",
               "B",
               "kappa_Intercept",
               "kappa_C",
               "kappa_B",
               "kappa_B:C",
           ])
```

```
        "1|Individual_kappa",
        "C|Individual_kappa",
        "B|Individual_kappa",
        "kappa_1|Individual_sigma",
        "kappa_C|Individual_sigma",
        "kappa_B|Individual_sigma",
        "kappa_B:C|Individual_sigma",
    ],
    cvars = ["Intercept",
             "C",
             "B",
             ]
)
```



5.11 Save all models

We may need these later, so we'll save them together with the same naming scheme.

```
[63]: import os
import arviz as az

# Ensure the results directory exists before saving
os.makedirs("2Results/unwrap_models", exist_ok=True)

# Dictionary mapping our fitted InferenceData objects to their descriptive_
↪ filenames
models_to_save = {
    "model_0_null": idata_null,
    "model_1_UV": idata_UV,
    "model_2_dim": idata_dim,
    "model_3_UV_dimKappa": idata_UV_dimKappa,
    "model_4_dim_UVKappa": idata_dim_UVKappa,
    "model_5_additive": idata_additive,
    "model_6_additive_interactionKappa": idata_additive_interactionKappa,
    "model_7_interaction": idata_interaction # Already fitted
}

# Systematically export each model to NetCDF format
for file_prefix, idata_obj in models_to_save.items():
    # Construct the clear, structured path string
    save_path = f"2Results/unwrap_models/GUV_unwrap_{file_prefix}.nc"

    print(f"Exporting: {save_path}...")
    az.to_netcdf(idata_obj, save_path)
```

```
Exporting: 2Results/unwrap_models/GUV_unwrap_model_0_null.nc...
Exporting: 2Results/unwrap_models/GUV_unwrap_model_1_UV.nc...
Exporting: 2Results/unwrap_models/GUV_unwrap_model_2_dim.nc...
Exporting: 2Results/unwrap_models/GUV_unwrap_model_3_UV_dimKappa.nc...
Exporting: 2Results/unwrap_models/GUV_unwrap_model_4_dim_UVKappa.nc...
Exporting: 2Results/unwrap_models/GUV_unwrap_model_5_additive.nc...
Exporting:
2Results/unwrap_models/GUV_unwrap_model_6_additive_interactionKappa.nc...
Exporting: 2Results/unwrap_models/GUV_unwrap_model_7_interaction.nc...
```

6 Model comparison for hypothesis testing

These models represent different hypotheses about the underlying causes of the data. We can test these hypotheses by comparing predictive power.

6.1 LOO-CV

Leave one out cross validation (LOO-CV) to calculate estimated log predictive density for each model (ELPD).

```
[64]: # Calculate predictive power using LOO cross-validation
import pymc as pm
import arviz as az

# Manually compute and attach the log-likelihood group to your existing idata
model_null.compute_log_likelihood(idata_null)
model_UV.compute_log_likelihood(idata_UV)
model_dim.compute_log_likelihood(idata_dim)
model_UV_dimKappa.compute_log_likelihood(idata_UV_dimKappa)
model_dim_UVKappa.compute_log_likelihood(idata_dim_UVKappa)
model_additive.compute_log_likelihood(idata_additive)
model_additive_interactionKappa.
    ↪compute_log_likelihood(idata_additive_interactionKappa)
model_interaction.compute_log_likelihood(idata_interaction)

# Compute LOO
loo_null = az.loo(idata_null)
loo_UV = az.loo(idata_UV)
loo_dim = az.loo(idata_dim)
loo_UV_dimKappa = az.loo(idata_UV_dimKappa)
loo_dim_UVKappa = az.loo(idata_dim_UVKappa)
loo_additive = az.loo(idata_additive)
loo_additive_interactionKappa = az.loo(idata_additive_interactionKappa)
loo_interaction = az.loo(idata_interaction)
```

```
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/arviz/stats/stats.py:792: UserWarning: Estimated shape parameter of
Pareto distribution is greater than 0.70 for one or more samples. You should
consider using a more robust model, this is because importance sampling is less
likely to work well if the marginal posterior and LOO posterior are very
different. This is more likely to happen with a non-robust model and highly
influential observations.
```

```
warnings.warn(
```

```
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/arviz/stats/stats.py:792: UserWarning: Estimated shape parameter of
Pareto distribution is greater than 0.70 for one or more samples. You should
consider using a more robust model, this is because importance sampling is less
likely to work well if the marginal posterior and LOO posterior are very
different. This is more likely to happen with a non-robust model and highly
influential observations.
```

```
warnings.warn(
```

```
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/arviz/stats/stats.py:1039: RuntimeWarning: overflow encountered in exp
```

```

    weights = 1 / np.exp(len_scale - len_scale[:, None]).sum(axis=1)
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/numpy/core/_methods.py:49: RuntimeWarning: overflow encountered in
reduce
    return umr_sum(a, axis, dtype, out, keepdims, initial, where)
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/arviz/stats/stats.py:792: UserWarning: Estimated shape parameter of
Pareto distribution is greater than 0.70 for one or more samples. You should
consider using a more robust model, this is because importance sampling is less
likely to work well if the marginal posterior and LOO posterior are very
different. This is more likely to happen with a non-robust model and highly
influential observations.
    warnings.warn(
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/arviz/stats/stats.py:792: UserWarning: Estimated shape parameter of
Pareto distribution is greater than 0.70 for one or more samples. You should
consider using a more robust model, this is because importance sampling is less
likely to work well if the marginal posterior and LOO posterior are very
different. This is more likely to happen with a non-robust model and highly
influential observations.
    warnings.warn(
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/arviz/stats/stats.py:792: UserWarning: Estimated shape parameter of
Pareto distribution is greater than 0.70 for one or more samples. You should
consider using a more robust model, this is because importance sampling is less
likely to work well if the marginal posterior and LOO posterior are very
different. This is more likely to happen with a non-robust model and highly
influential observations.
    warnings.warn(
/opt/miniconda3/envs/nutpie-env/lib/python3.12/site-
packages/arviz/stats/stats.py:792: UserWarning: Estimated shape parameter of
Pareto distribution is greater than 0.70 for one or more samples. You should
consider using a more robust model, this is because importance sampling is less
likely to work well if the marginal posterior and LOO posterior are very
different. This is more likely to happen with a non-robust model and highly
influential observations.
    warnings.warn(

```



```
[65]: # Print the results
print('\n----- Model 0: no effects ----- \n')
print(loo_null)

print('\n----- Model 1: UV (C only) ----- \n')
print(loo_UV)

print('\n----- Model 2: Dim (B only) ----- \n')
print(loo_dim)

print('\n----- Model 3: UV angle, Dim+UV kappa ----- \n')
print(loo_UV_dimKappa)

print('\n----- Model 4: Dim angle, Dim+UV kappa ----- \n')
print(loo_dim_UVKappa)

print('\n----- Model 5: Additive (B + C) ----- \n')
print(loo_additive)

print('\n----- Model 6: Kappa interaction (B * C) ----- \n')
print(loo_additive_interactionKappa)

print('\n----- Model 7: Full interaction (B * C) ----- \n')
print(loo_interaction)
```

----- Model 0: no effects -----

Computed from 8000 posterior samples and 6231 observations log-likelihood matrix.

	Estimate	SE
elpd_loo	-9257.40	55.57
p_loo	298.71	-

Pareto k diagnostic values:

		Count	Pct.
(-Inf, 0.70]	(good)	6231	100.0%
(0.70, 1]	(bad)	0	0.0%
(1, Inf)	(very bad)	0	0.0%

----- Model 1: UV (C only) -----

Computed from 8000 posterior samples and 6231 observations log-likelihood matrix.

	Estimate	SE
elpd_loo	-9074.42	56.64
p_loo	365.59	-

There has been a warning during the calculation. Please check the results.

Pareto k diagnostic values:

		Count	Pct.
(-Inf, 0.70]	(good)	6227	99.9%
(0.70, 1]	(bad)	4	0.1%
(1, Inf)	(very bad)	0	0.0%

----- Model 2: Dim (B only) -----

Computed from 8000 posterior samples and 6231 observations log-likelihood matrix.

	Estimate	SE
elpd_loo	-6243.32	74.05
p_loo	515.02	-

There has been a warning during the calculation. Please check the results.

Pareto k diagnostic values:

		Count	Pct.
(-Inf, 0.70]	(good)	6191	99.4%
(0.70, 1]	(bad)	22	0.4%
(1, Inf)	(very bad)	18	0.3%

----- Model 3: UV angle, Dim+UV kappa -----

Computed from 8000 posterior samples and 6231 observations log-likelihood matrix.

	Estimate	SE
elpd_loo	-7041.28	72.18
p_loo	484.17	-

There has been a warning during the calculation. Please check the results.

Pareto k diagnostic values:

		Count	Pct.
(-Inf, 0.70]	(good)	5857	94.0%

(0.70, 1]	(bad)	85	1.4%
(1, Inf)	(very bad)	289	4.6%

----- Model 4: Dim angle, Dim+UV kappa -----

Computed from 8000 posterior samples and 6231 observations log-likelihood matrix.

	Estimate	SE
elpd_loo	-6132.68	74.22
p_loo	559.44	-

There has been a warning during the calculation. Please check the results.

Pareto k diagnostic values:

		Count	Pct.
(-Inf, 0.70]	(good)	6159	98.8%
(0.70, 1]	(bad)	34	0.5%
(1, Inf)	(very bad)	38	0.6%

----- Model 5: Additive (B + C) -----

Computed from 8000 posterior samples and 6231 observations log-likelihood matrix.

	Estimate	SE
elpd_loo	-6111.42	74.33
p_loo	582.95	-

There has been a warning during the calculation. Please check the results.

Pareto k diagnostic values:

		Count	Pct.
(-Inf, 0.70]	(good)	6165	98.9%
(0.70, 1]	(bad)	32	0.5%
(1, Inf)	(very bad)	34	0.5%

----- Model 6: Kappa interaction (B * C) -----

Computed from 8000 posterior samples and 6231 observations log-likelihood matrix.

Estimate	SE
----------	----

```
elpd_loo -6104.66    74.48
p_loo      589.48      -
```

There has been a warning during the calculation. Please check the results.

Pareto k diagnostic values:

		Count	Pct.
(-Inf, 0.70]	(good)	6144	98.6%
(0.70, 1]	(bad)	51	0.8%
(1, Inf)	(very bad)	36	0.6%

----- Model 7: Full interaction (B * C) -----

Computed from 8000 posterior samples and 6231 observations log-likelihood matrix.

```
      Estimate      SE
elpd_loo -6048.45    74.41
p_loo      593.35      -
```

There has been a warning during the calculation. Please check the results.

Pareto k diagnostic values:

		Count	Pct.
(-Inf, 0.70]	(good)	6187	99.3%
(0.70, 1]	(bad)	34	0.5%
(1, Inf)	(very bad)	10	0.2%

6.2 Compare models

```
[66]: # and set precision to 4 places
np.set_printoptions(precision=4)#, suppress=True)#Suppress scientific notation

# Compare, ranking models from best to worst
comparison = az.compare(
    {
        'No effects': loo_null,
        'UV (C)': loo_UV,
        'Dim (B)': loo_dim,
        'UV_dimKappa': loo_UV_dimKappa,
        'Dim_UVKappa': loo_dim_UVKappa,
        'Additive': loo_additive,
        'Additive_interKappa': loo_additive_interactionKappa,
```

```

    'Interaction': loo_interaction
  },
  ic="loo")

# Compare specifically effect of brightness
comparison_bright = az.compare({
  'No effects': loo_null,
  'Brightness': loo_dim,
},
  ic="loo",
)

# Compare specifically effect of colour
comparison_colour = az.compare({
  'No effects': loo_null,
  'Colour': loo_UV,
},
  ic="loo",
)

# Compare specifically effect of green dim turn
comparison_dimG = az.compare({
  'Brightness + kappa colour': loo_UV_dimKappa,
  'Brightness + colour': loo_additive,
},
  ic="loo",
)

print('\n----- Model Comparison Ranking ----- \n')
print(comparison.round(2))

print('\n----- Brightness effect ----- \n')
print(comparison_bright.round(2))

print('\n----- Colour effect ----- \n')
print(comparison_colour.round(2))

print('\n----- Brightness effect on \u03bc ----- \n')
print(comparison_dimG.round(2))

```

----- Model Comparison Ranking -----

	rank	elpd_loo	p_loo	elpd_diff	weight	se	dse	\
Interaction	0	-6048.45	593.35	0.00	0.83	74.41	0.00	
Additive_interKappa	1	-6104.66	589.48	56.21	0.00	74.48	12.92	
Additive	2	-6111.42	582.95	62.97	0.13	74.33	14.28	
Dim_UVKappa	3	-6132.68	559.44	84.22	0.00	74.22	15.55	
Dim (B)	4	-6243.32	515.02	194.87	0.00	74.05	21.13	
UV_dimKappa	5	-7041.28	484.17	992.83	0.03	72.18	45.00	

UV (C)	6	-9074.42	365.59	3025.97	0.00	56.64	61.66
No effects	7	-9257.40	298.71	3208.95	0.00	55.57	62.59

	warning	scale
Interaction	True	log
Additive_interKappa	True	log
Additive	True	log
Dim_UVKappa	True	log
Dim (B)	True	log
UV_dimKappa	True	log
UV (C)	True	log
No effects	False	log

----- Brightness effect -----

	rank	elpd_loo	p_loo	elpd_diff	weight	se	dse	warning	\
Brightness	0	-6243.32	515.02	0.00	1.0	74.05	0.00	True	
No effects	1	-9257.40	298.71	3014.08	0.0	55.57	60.99	False	

	scale
Brightness	log
No effects	log

----- Colour effect -----

	rank	elpd_loo	p_loo	elpd_diff	weight	se	dse	warning	\
Colour	0	-9074.42	365.59	0.00	1.0	56.64	0.00	True	
No effects	1	-9257.40	298.71	182.98	0.0	55.57	15.58	False	

	scale
Colour	log
No effects	log

----- Brightness effect on -----

	rank	elpd_loo	p_loo	elpd_diff	weight	se	\
Brightness + colour	0	-6111.42	582.95	0.00	0.96	74.33	
Brightness + kappa colour	1	-7041.28	484.17	929.86	0.04	72.18	

	dse	warning	scale
Brightness + colour	0.00	True	log
Brightness + kappa colour	44.19	True	log

6.3 Plot model comparison

Plot relative and contrasted ELPD for the models of interest.

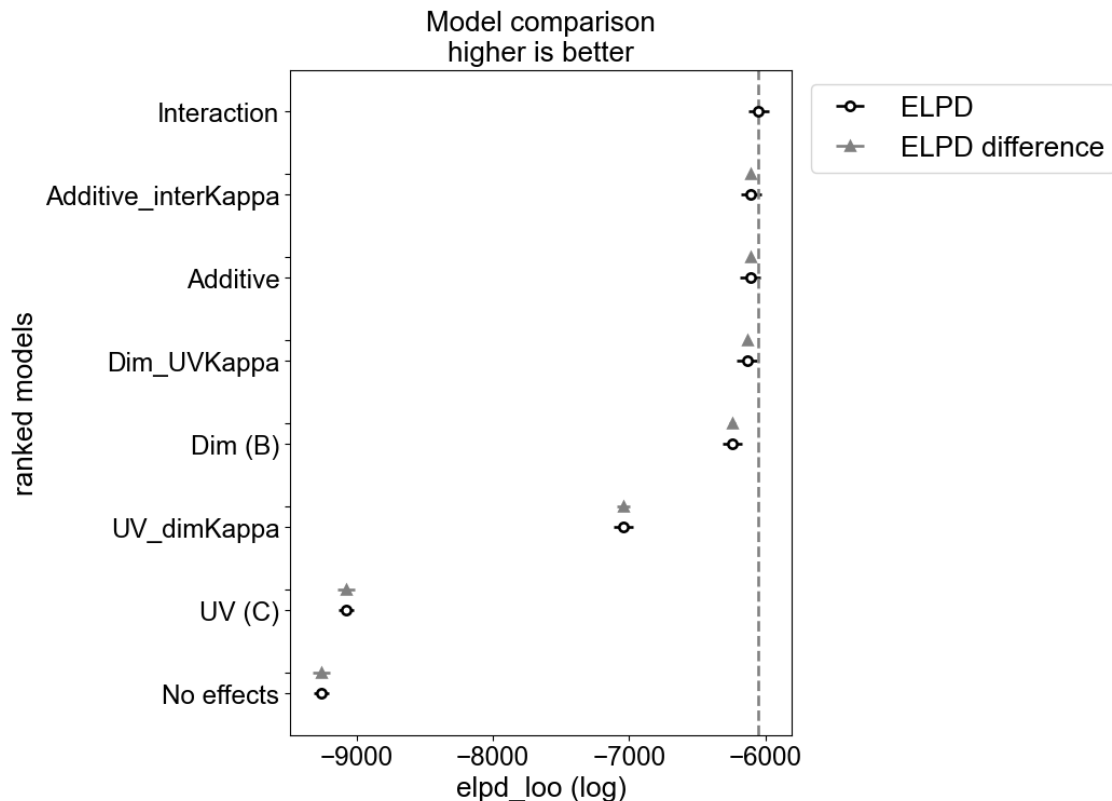
```
[67]: import matplotlib as mpl

# Ensure text is output as editable TrueType text elements in PDFs
mpl.rcParams['pdf.fonttype'] = 42
mpl.rcParams['ps.fonttype'] = 42

# Optional: Use standard Arial/Helvetica to guarantee font compatibility across
# systems
mpl.rcParams['font.family'] = 'sans-serif'
mpl.rcParams['font.sans-serif'] = ['Arial', 'Helvetica', 'Computer Modern Sans-
Serif']

# Visualize the comparison
ax_compare = az.plot_compare(comparison)

# Extract the parent figure belonging to that specific axis
fig_compare = ax_compare.get_figure()
fig_compare.savefig("0Figures/model_comparison.pdf", format="pdf",
bbox_inches="tight")
```



7 Extract & inspect posterior samples from maximal model

```
[68]: import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import vonmises, circmean
from unwrap.link.numpy import shifted_modulo, softplus

# -----
# Extract posterior samples for population-level parameters
# -----
mu_intercept = idata_interaction.posterior['Intercept'].values
mu_slopeB    = idata_interaction.posterior['B'].values
mu_slopeC    = idata_interaction.posterior['C'].values
mu_slopeBC   = idata_interaction.posterior['B:C'].values

kappa_intercept = idata_interaction.posterior['kappa_Intercept'].values
kappa_slopeB    = idata_interaction.posterior['kappa_B'].values
kappa_slopeC    = idata_interaction.posterior['kappa_C'].values
kappa_slopeBC   = idata_interaction.posterior['kappa_B:C'].values

# -----
# Prepare data grid
# -----
theta = np.linspace(-np.pi, np.pi, 200)
theta_deg = theta * 180 / np.pi
num_samples = mu_intercept.flatten().shape[0]

# Compute population-level parameters for each condition
mu_pop_gh_samples_all = shifted_modulo(mu_intercept.flatten())
mu_pop_gl_samples_all = shifted_modulo((mu_intercept + mu_slopeB).flatten())
mu_pop_uh_samples_all = shifted_modulo((mu_intercept + mu_slopeC).flatten())
mu_pop_ul_samples_all = shifted_modulo((mu_intercept + mu_slopeB + mu_slopeC +
    ↪mu_slopeBC).flatten())

kappa_pop_gh_samples_all = softplus(kappa_intercept.flatten())
kappa_pop_gl_samples_all = softplus((kappa_intercept + kappa_slopeB).flatten())
kappa_pop_uh_samples_all = softplus(kappa_intercept.flatten() + kappa_slopeC.
    ↪flatten()) # Fixed: Match formula syntax safely
kappa_pop_ul_samples_all = softplus((kappa_intercept + kappa_slopeB +
    ↪kappa_slopeC + kappa_slopeBC).flatten())

# -----
# Compute von Mises PDFs
# -----
pdf_gh = vonmises.pdf(theta[None, :], kappa=kappa_pop_gh_samples_all[:, None],
    ↪loc=mu_pop_gh_samples_all[:, None])
```



```

pdf_gl = vonmises.pdf(theta[None, :], kappa=kappa_pop_gl_samples_all[:, None],
    ↪loc=mu_pop_gl_samples_all[:, None])
pdf_uh = vonmises.pdf(theta[None, :], kappa=kappa_pop_uh_samples_all[:, None],
    ↪loc=mu_pop_uh_samples_all[:, None])
pdf_ul = vonmises.pdf(theta[None, :], kappa=kappa_pop_ul_samples_all[:, None],
    ↪loc=mu_pop_ul_samples_all[:, None])

# Calculate medians and true 95% Credible Intervals (2.5% to 97.5%)
median_pdf_gh = np.median(pdf_gh, axis=0)
lower_pdf_gh = np.percentile(pdf_gh, 2.5, axis=0)
upper_pdf_gh = np.percentile(pdf_gh, 97.5, axis=0)

median_pdf_gl = np.median(pdf_gl, axis=0)
lower_pdf_gl = np.percentile(pdf_gl, 2.5, axis=0)
upper_pdf_gl = np.percentile(pdf_gl, 97.5, axis=0)

median_pdf_uh = np.median(pdf_uh, axis=0)
lower_pdf_uh = np.percentile(pdf_uh, 2.5, axis=0)
upper_pdf_uh = np.percentile(pdf_uh, 97.5, axis=0)

median_pdf_ul = np.median(pdf_ul, axis=0)
lower_pdf_ul = np.percentile(pdf_ul, 2.5, axis=0)
upper_pdf_ul = np.percentile(pdf_ul, 97.5, axis=0)

# -----
# Setup 2x2 Subplot Grid with Hex Color Codes
# -----
fig, axes = plt.subplots(2, 2, figsize=(15, 12), sharex=True, sharey=True)
((ax1, ax2), (ax3, ax4)) = axes

conditions = [
    {"ax": ax1, "med": median_pdf_gh, "low": lower_pdf_gh, "up": upper_pdf_gh,
    ↪"color": "#00CD00", "title": "Green Bright (B=0, C=0)", "b_val": 0, "c_val": 0},
    ↪},
    {"ax": ax2, "med": median_pdf_gl, "low": lower_pdf_gl, "up": upper_pdf_gl,
    ↪"color": "#008B00", "title": "Green Dim (B=1, C=0)", "b_val": 1, "c_val": 0},
    ↪},
    {"ax": ax3, "med": median_pdf_uh, "low": lower_pdf_uh, "up": upper_pdf_uh,
    ↪"color": "#CD00CD", "title": "UV Bright (B=0, C=1)", "b_val": 0, "c_val": 1},
    ↪},
    {"ax": ax4, "med": median_pdf_ul, "low": lower_pdf_ul, "up": upper_pdf_ul,
    ↪"color": "#A020F0", "title": "UV Dim (B=1, C=1)", "b_val": 1, "c_val": 1}
]

# Plot density and intervals per panel using updated colors
for cond in conditions:
    ax = cond["ax"]

```

```

    ax.plot(theta_deg, cond["med"], color=cond["color"], lw=2.5,
    ↪label="Posterior Median")
    ax.fill_between(theta_deg, cond["low"], cond["up"], color=cond["color"],
    ↪alpha=0.25, label="95% Credible Interval")
    ax.set_title(cond["title"], fontsize=12, fontweight="bold")
    ax.grid(True, alpha=0.3)

# -----
# Calculate and Overlap Individual Summary Means
# -----
for i, individual in enumerate(df['Individual'].unique()):
    individual_data = df[df['Individual'] == individual]

    for cond in conditions:
        # Extract observations specific to this condition panel
        headings = individual_data.loc[(individual_data['B'] == cond["b_val"])
    ↪& (individual_data['C'] == cond["c_val"]), 'angle'].dropna()

        if len(headings) > 0:
            ind_circmean = shifted_modulo(circmean(headings))
            cond["ax"].scatter(
                ind_circmean * 180 / np.pi, 0,
                color=cond["color"], edgecolor='black', linewidth=0.5,
                s=40, alpha=0.6,
                label='Individual Means' if i == 0 else ""
            )

# Global formatting details
for ax in axes.flat:
    ax.set_xlabel('Heading (°)')
    ax.set_ylabel('Density')
    ax.label_outer() # Hides inner labels to avoid overlap clutter
    ax.legend(loc="upper right", frameon=True, facecolor="white", framealpha=0.
    ↪8)

plt.suptitle('von Mises Probability Density', fontsize=16, fontweight="bold",
    ↪y=0.96)
plt.tight_layout()
plt.show()

```

